Strategy and model-level optimality in *Cookie Clicker*

Claude (Anthropic)
for jehlp.net and the Cookie Monster+ project

October 2026

Abstract

We study purchase order, banking, heavenly upgrades, and ascension in *Cookie Clicker*. The requested objective is minimum expected completion time from a new save. Completion includes all normal achievements, upgrades, and unlocks. Shadow achievements and specified long sugar-lump time gates are excluded.

The existing autoplay policy does not have a proof of this objective. We give a strict counterexample to its greedy purchase rule, even with constant additive production. We correct achievement valuation, heavenly-block scheduling, and the ascension planner's missing no-reset action. We also correct two execution defects found in the installed game source.

A finite purchase model admits an exact completion policy by dynamic programming. The accompanying Lean project checks explicitly stated model results and a completion predicate generated from the pinned Steam game. An isolated adapter executes the installed game source. Its legal action coverage and its correspondence to Lean are not complete. The remaining gameplay policies are heuristics. Historical simulation results are retained as unverified reports, not certificates of optimality.

MSC 2020: 90C39 (primary); 60G40, 90B35 (secondary).
Keywords: incremental games, optimal stopping, dynamic programming, exchange arguments, scheduling.

Contents

1 Introduction **3**
1.1 The four rules . . . 3
1.2 Scope and conventions . . . 3
1.3 The completion objective . . . 4
1.4 A retained time gate and controllable random state . . . 5
1.5 A native zero-delay acquisition . . . 7
2 Model **7**
2.1 Mechanics . . . 7
2.2 Notation . . . 8
2.3 Assumptions . . . 8
3 Purchase order **8**
3.1 An exact finite completion model . . . 9
4 Golden cookies and the bank **10**
4.1 When to hold a "Lucky!" bank . . . 10
4.2 Values under automation . . . 11

5 Heavenly chips **12**

6 When to ascend **15**

6.1 Time rescaling . . . 15
6.2 Formulation . . . 15
6.3 Late game: a closed form . . . 15
6.4 Early game: why early resets fail . . . 17
6.5 The general planner . . . 18

7 The rest of the game **19**

7.1 Achievements . . . 19
7.2 Godzamok . . . 20
7.3 Wrinklers and wrath cookies . . . 22
7.4 Minigames . . . 23
7.5 The dragon, seasons, sugar lumps and permanent slots . . . 26

8 Historical numerical reports **27**

9 Implementation **28**

10 Formal verification and audit scope **29**

11 Limitations **52**

A Algorithms **53**

References **54**

1 Introduction

Cookie Clicker [8] is an incremental game. Clicks and buildings produce cookies. Each copy of a building costs 1.15 times the previous copy. Upgrades multiply production, and random *golden cookies* give temporary bonuses.

At any time the player can *ascend*. Ascension abandons the current run. In return the player gains *prestige*, a permanent production multiplier that grows like the cube root of all cookies ever baked. Prestige also gives *heavenly chips*, which buy permanent upgrades.

Four decisions recur in play: purchases, golden-cookie banking, heavenly upgrades, and ascension. This paper separates exact model results from gameplay estimates. The audit of 29 October 2026 corrects claims that exceeded their assumptions.

1.1 The four rules

Let c be the income rate. Item i costs p_i and adds Δ_i to the income.

1. *Two-purchase order* (Theorem 3.1). For two required purchases with neither affordable, compare the index
$$E_i = \frac{p_i}{c} + \frac{p_i}{\Delta_i}.$$

2. *The* "Lucky!" *bank* (Proposition 4.1). The frozen marginal-income comparison favours item i if and only if
$$\frac{p_i}{\bar{\Delta}_i} < \theta = \frac{1}{0.15\, g\, \lambda_L}.$$
Here $\bar{\Delta}_i$ is the full income gain of the item, g multiplies golden-cookie gains and λ_L is the rate of "Lucky!" cookies.

3. *Heavenly chips* (Theorem 5.1). Assume constant chip effort, no initial chip bank, and no parent constraints. Upgrade j costs p_j chips and multiplies speed by m_j. To acquire a fixed set, use decreasing order of
$$\rho_j = \frac{1 - 1/m_j}{p_j}.$$

4. *Ascension* (Corollary 6.3). In the power-law model with zero reset overhead, a step attaining the lower bound uses the root $v^\star > 1$ of
$$\frac{\beta}{3}\Big(1 - \frac{1}{v}\Big) = \frac{1 - v^{-\gamma/3}}{\gamma}, \qquad \gamma = \frac{3}{\beta} - 1,$$
where β is the growth exponent of the current run.

Section 7 adds rules for the other systems: achievements, Godzamok, wrinklers, the minigames, the dragon and seasons. Section 8 reports historical simulator results whose original traces are unavailable.

1.2 Scope and conventions

A local income comparison does not prove minimum completion time. The word "optimal" below refers only to the model and objective stated with each result. Cookie Monster+ [3] combines these comparisons with estimated values and execution rules. It is not a certified solution to the full completion problem.

1.3 The completion objective

Let s_0 be a new save, let π be a legal policy, and let $\mathcal{T}$ be the completion set. The requested objective is

$$\inf_{\pi} \mathbb{E}^{\pi}_{s_0}[\tau_{\mathcal{T}}], \qquad \tau_{\mathcal{T}} = \inf\{t \geq 0 : s_t \in \mathcal{T}\}.$$

The set $\mathcal{T}$ requires all normal achievements, upgrades, and finite unlocks under the versioned profile in `model/completion-profile.json`. Shadow achievements are excluded. Completion cannot mean merely "nothing is affordable now". Repeated building purchases and levels are not terminal requirements in their own right.

The initialized Steam 2.053 catalogue contains 622 normal achievements, 17 shadow achievements, four unused dungeon achievements, and 875 upgrades. The profile excludes 24 individual sugar achievements: the 20 building-level achievements, the 365-lump total, and the normal bifurcated, meaty, and caramelized lump achievements. All other 598 normal achievements remain required. The profile retains lump costs for ordinary progression. Cursor level 12, farm level 9, and the four minigame unlocks still apply.

The terminal predicate requires current ownership of all 845 legal non-toggle upgrades, including heavenly, cosmetic, and offline upgrades. It requires access to all 17 toggles and selectors; matching-version acquisition history can satisfy mutually exclusive choices. All 34 Garden seeds must currently be unlocked. Thus the required Garden sacrifice must be followed by a complete seed log again. The final dragon, Santa, office, plot, soil, spell, god, good, and loan unlocks also count. The final state must be in a normal run, outside ascension and reincarnation transitions.

Completion count readers now admit every represented nonnegative finite integer, including values at and above 2^{53}. Loss of unit-increment precision does not invalidate a native threshold comparison. Lean checks the count conversion at 2^{53}, 2^{1000} and the largest finite binary64 integer, $2^{1024} - 2^{971}$. JSON observations use binary64 decimal conversion before the count check; nonfinite, negative and fractional counts remain invalid. These reader results do not assert that all supplied large-count fixtures are reachable. The source-heap reader also rejects catalogue descriptors explicitly marked unresolved, including owned upgrades, spells and gods. It does not replace an unknown descriptor with a recorded placeholder.

Every milk choice is a required unlock. Hazelnut milk requires 600 normal achievements. The 598 specified achievements and the achievement implied by cursor level 12 supply only 599. At least one other excluded sugar achievement must therefore be won, unless it has already been obtained. This does not require all 24 excluded achievements. It preserves a separate finite unlock dependency. Shadow achievements do not contribute to this count. The predicate also requires the actual native `AchievementsOwned` counter to reach 600, because the milk selector reads that counter. It does not infer the counter from won flags. A source callback can set a flag before later notification work and the counter increment; an interrupted callback can therefore leave the two values different.

The executable predicate reports every missing condition. It does not prove that an arbitrary supplied state or acquisition history is reachable. The current mod omits several required achievement and seed paths and still uses income forecasts. Random events need state-dependent transition probabilities; the aggregate outcome shares in this paper do not supply them. A complete proof also needs an attaining policy and a matching expected-time lower bound. This audit does not supply that full-game certificate.

The user permits scheduled maintenance reloads that save the current state and then reload it. The user excludes earlier-save restoration, save/reload outcome retries, and deliberate random-outcome manipulation through native UI actions. The user also permits reading and decoding the current save shown by native Export save, including the startup seed. Native

export and save bookkeeping effects remain part of the transition. This does not permit import, editing, restoration, or retries. The user forbids timing chosen to change seeded outcomes, including harvest timestamps selected using the exported seed. This restriction concerns the decision rule: the same timestamp can result from an ordinary schedule or a seed search, so a trace alone cannot distinguish them. The strategy must not change the clock or inspect future free draws or generator state that the native public interface does not reveal. Diagnostic fixtures used to audit such behavior are not admissible strategies. These restrictions do not remove the need to specify the transition law under permitted observations.

The selected mathematical environment supplies independent uniform free draws on $\{k/2^{53} : 0 \le k < 2^{53}\}$. Explicitly seeded generators retain the pinned native algorithm, seed construction, and draw order. A no-argument `Math.seedrandom()` returns to the independent free law; this is an environment idealization, not a statement about browser entropy. The clock is monotone and unbounded. Finite native Date range and eventual floating-point timestamp stagnation are therefore not completion obstructions in this mathematical model. The selected extension also processes arbitrary finite timestamp text without native string-limit failure or a stuck traversal index, while preserving native seeded results for ordinary timestamps. Ordinary economic arithmetic remains binary64. Source propagation of this extension remains a proof obligation. The executable experiments still use finite native dates and a reproducible sample generator. They do not implement the full selected probability space or unbounded calendar.

The user also fixes the herald count at 100, with no later external changes. This is an explicit environment assumption: the Steam source otherwise obtains player-count replies from an external service, whose future law is not specified by the game. The isolated adapter already supplies a constant reply that the native callback maps to 100 heralds; policy evaluation now rejects another count. The source callback and its scheduling still require formal refinement.

Times are in seconds unless a unit is given. All logarithms are natural. The game runs at 30 frames per second. A *run* is the period between two ascensions, and *CpS* means cookies per second from buildings.

1.4 A retained time gate and controllable random state

Hand-picked remains required. Its source branch requires a mature but unripe natural lump. With all permitted maturity reductions, the maturity threshold in milliseconds is

$$d = \frac{20 \cdot 3600000 - 3600000 - 7 \cdot 60000}{1 + 0.05 \cdot 1.1} = \frac{13\,596\,000\,000}{211}.$$

The reductions are Diabetica Daemonicus, Ichor syrup, Dragon's Curve, and Reality Bending. The integer clock must therefore advance at least $\lceil d \rceil = 64\,436\,019$ milliseconds from the lump's start before that branch is eligible. This is about 17 hours 54 minutes. It omits the time needed to unlock the lump and obtain the reductions. Lean checks the rational arithmetic and integer threshold. Native-engine tests check the corresponding numeric boundary. These are separate checks; they do not prove a refinement of the whole engine.

A tempting expected-time bound is $2d$, obtained by treating every attempt as a fresh fair coin and restarting a full growth interval after failure. That conclusion needs additional assumptions. Ordinary save rollback can undo an unpersisted failed attempt. More subtly, the You building's clone-hover renderer seeds the global generator for a clone name and does not restore it before returning. A later harvest can therefore use the seeded stream. The audit reproduces this callback behavior in the pinned engine. Both forms of outcome manipulation

are excluded from the requested strategy class. The diagnostic fixture is not an admissible new-save route, and excluding it still does not prove a conditional fair-coin law for the remaining controls.

The selected free law is more specific than ECMAScript, which specifies an approximately uniform random or pseudorandom result rather than an exact independent law [9]. The game's own seeded calls add observable dependencies. Replacing those seeded calls with independent draws would change the control problem. A source inventory accounts for 23 reseeding call sites in eight callback bodies. The Lean control-flow analysis includes branches, early returns, loops, and seven expanded source helpers. Unresolved calls have an unknown phase; they are not assumed to preserve it. Three extracted callback closures have no unresolved explicit calls. The forbidden clone-hover leak is detected. The source extractor, implicit exceptions, host operations, and the remaining callees still require justification before this is a whole-engine phase theorem.

Even the five-letter startup seed is not exactly uniform over the 26^5 strings. The native expression is `floor(26*Math.random())`, with a binary64 product. Put $q = 346\,430\,740\,566\,961$. The letters a, f, k, o, u, and y each have $q + 1$ of the 2^{53} input indices; each other letter has q. Lean proves all 26 thresholds for an integer projection of that rounded product and the resulting five-letter product law. Native boundary comparisons and actual seed-generation calls agree. A uniform prior over seed strings would therefore change the selected probability model.

Public observations can reveal a seed without reading hidden state. The visible fortune numbers $(73, 43, 10, 64)$ leave exactly two startup candidates, `aaodk` and `abeby`, in an exhaustive scan of all 11 881 376 strings. Their prior weights differ. The visible Mice clicking mice tooltip then distinguishes these candidates by its seeded wording, "blasphemouse" or "blasphemous". The scan and its source hashes are archived. This is a finite inference witness, not a theorem that every seed is identifiable. A legal history-dependent policy must nevertheless be allowed to retain such information. Seeded spell outcomes cannot in general be replaced by fresh independent outcomes after public observations.

For finite adaptive trees built from the selected free kernel, Lean computes the two-item harvest choice probability as exactly $1/2$. Arbitrary intervening free observations and decisions to stop do not improve the probability of success: after at most n harvest attempts, failure has probability at least 2^{-n}. The theorem derives this inequality from the kernel instead of assuming a conditional-fairness field. Applying it to arbitrary game histories still requires the source-phase correspondence and the infinite-history probability construction.

A conditional lower bound can nevertheless be stated precisely. Let $D = 64\,436\,019$ be the minimum integer spacing. Assume that Hand-picked is initially absent, each attempted unripe harvest has conditional success probability at most $1/2$, and every next eligible attempt requires at least D milliseconds after a failed attempt. The first eligible attempt must also require at least D milliseconds from the new save. If T is the full completion time, with $T = \infty$ on unsuccessful paths, then for every integer $n \geq 0$,

$$\mathbb{E}[T] \;\geq\; D \sum_{j=0}^{n-1} 2^{-j} = 2D(1 - 2^{-n}).$$

The first D integer survival terms are one. Each following block is bounded below by the probability of still lacking Hand-picked after the preceding attempts. The conditional failure bound gives at least 2^{-j} for block j. Taking the supremum over n yields $\mathbb{E}[T] \geq 128\,872\,038$ milliseconds, about 35.8 hours. This is a necessary bound under the stated bridge assumptions. It is not attained by any policy established here, and it omits other mandatory goals.

1.5 A native zero-delay acquisition

Dragon petting has no gameplay cooldown. The two-second check in `Game.ClickSpecialPic` limits a sound; every pet makes another free drop trial. At a fixed quarter-hour, the native seeded shuffle chooses the same item. If that item is known to be missing, the selected 53-bit law gives the exact trial probability

$$p = \frac{450\,359\,962\,737\,050}{2^{53}} = \frac{225\,179\,981\,368\,525}{4\,503\,599\,627\,370\,496}.$$

This is slightly greater than 1/20 because the native comparison uses the binary64 value of `1/20` and a finite uniform grid. After n failures the remaining mass is $(1-p)^n > 0$, and this mass tends to zero. With zero simulated input latency, retrying until the visible unlock occurs therefore attains zero additional elapsed time almost surely. The infinite-failure path retains hitting time ∞ and has probability zero. Lean checks the exact count, normalized first-success masses, hitting-time convention, and local minimum. This result concerns one known-missing quarter's item, not full-game completion.

The missing-item condition matters. A quarter can select an item already owned, in which case indefinite same-time retries do not help. The experimental controller learns quarter-to-item mappings only from observed new unlocks. It skips known completed quarters and uses positive waits when existing ownership makes the current quarter uncertain. The practical mod uses bounded batches to keep the interface responsive. Version 2.053.16 removes its former one-minute batch delay, stops on an observed unlock, and preserves the user's menu and volume.

2 Model

2.1 Mechanics

A run starts with no buildings. Building k with n copies costs

$$p_k(n) = p_k(0)\,1.15^{\,n}. \tag{2.1}$$

Let A be the cookies baked in all previous runs, and C the cookies baked in the current run. Ascension converts all-time cookies into the prestige

$$P(A+C) = \Big(\frac{A+C}{10^{12}}\Big)^{1/3}. \tag{2.2}$$

Each prestige level adds h percent to production. The *prestige power* h becomes 1 after the heavenly upgrade *Legacy* (one chip) and five cheap store upgrades, *Heavenly chip secret* through *Heavenly key*. A run that starts at prestige P thus has the production multiplier

$$M(P) = 1 + 0.01\,h\,\lfloor P \rfloor\,. \tag{2.3}$$

The player receives $\lfloor P \rfloor$ heavenly chips over the whole game. Each heavenly upgrade has a price in chips and a set of parent upgrades, which must be owned first.

2.2 Notation

Symbol	Meaning
c	income rate (cookies per second)
b	cookies in the bank
$p_i,\ \Delta_i$	price of purchase i and the income it adds
$A,\ C$	cookies baked before the current run, and in it
$P,\ h$	prestige (2.2) and prestige power
S	speed of a run: the factor by which permanent bonuses multiply all income
$F(x)$	run curve: cookies baked after x units of speed-one time
β	local growth exponent of a run, $\beta = cT/C$
τ	time lost per ascension

2.3 Assumptions

Assumption A1. Between purchases the income rate is constant. Purchases are instantaneous.

Assumption A2. The income that a purchase adds does not depend on the order of a given pair of purchases.

Assumption A3. Permanent bonuses multiply every income source (buildings, clicks and golden-cookie rewards) by the same factor S.

Assumption A4. Purchase decisions depend on the bank and the holdings only through comparisons that do not change when all incomes are multiplied by a constant.

Assumption A5. (Section 6.3 only.) The run curve is a power law, $F(x) = \alpha x^{\beta}$ with $\beta > 1$.

3 Purchase order

Theorem 3.1 (Pairwise exchange)**.** *Assume A1 and A2, with $c, p_A, p_B, \Delta_A, \Delta_B > 0$. Two purchases A and B occur one after the other, from a bank $b \le \min(p_A, p_B)$ at income c. Buying A first completes both purchases no later than buying B first if and only if*

$$E_A \le E_B, \qquad E_i = \frac{p_i}{c} + \frac{p_i}{\Delta_i}. \tag{3.1}$$

Both orders end in the same state.

Proof. With A first, the bank reaches p_A after $(p_A - b)/c$ seconds and is then empty. Purchase B follows after a further $p_B/(c + \Delta_A)$ seconds. Thus

$$T_{AB} = \frac{p_A - b}{c} + \frac{p_B}{c + \Delta_A}, \qquad T_{BA} = \frac{p_B - b}{c} + \frac{p_A}{c + \Delta_B}.$$

The bank b cancels in the difference, and

$$\begin{aligned} T_{AB} \le T_{BA} &\iff p_A\Big(\frac{1}{c} - \frac{1}{c + \Delta_B}\Big) \le p_B\Big(\frac{1}{c} - \frac{1}{c + \Delta_A}\Big) \\ &\iff \frac{p_A\,\Delta_B}{c + \Delta_B} \le \frac{p_B\,\Delta_A}{c + \Delta_A} \\ &\iff \frac{p_A}{c} + \frac{p_A}{\Delta_A} \le \frac{p_B}{c} + \frac{p_B}{\Delta_B}. \end{aligned}$$

The last step multiplies by $(c + \Delta_A)(c + \Delta_B)/(c\,\Delta_A\Delta_B) > 0$. After both purchases, the holdings, the empty bank and the income $c + \Delta_A + \Delta_B$ agree by A2. □

Remark 3.2. Both orders end in the same state. Thus the faster order is preferable when these two purchases must come next. This proves an adjacent-pair comparison, not a globally optimal choice among three or more purchases. The index changes with income. The following counterexample has strict comparisons throughout.

Example 3.3 (A strict counterexample to greedy completion)**.** Start with $b = 0$ and $c = 1$. Three required items have (p, Δ) equal to $A = (3, 1)$, $B = (5, 4)$, and $C = (6, 6)$. Initially the indices are 6, 25/4, and 7. Greedy buys A. At income 2, the remaining indices are 15/4 and 4, so it buys B next. Its completion time is

$$T_{ABC} = 3 + \frac{5}{2} + \frac{6}{6} = \frac{13}{2}.$$

The order B, C, A takes

$$T_{BCA} = 5 + \frac{6}{5} + \frac{3}{11} = \frac{356}{55} = \frac{13}{2} - \frac{3}{110}.$$

Both orders finish with the same holdings, an empty bank, and income 12. Thus the greedy policy is slower under the paper's own assumptions.

3.1 An exact finite completion model

Fix a finite set R of required, independent, single-use purchases. Prices are nonnegative and gains are nonnegative. Income c is positive, the bank b is nonnegative, and no random event or reset occurs. Completion means owning every item in R.

For each remaining item i, define the earliest waiting time and resulting bank by

$$w_i(b, c) = \frac{\max(p_i - b, 0)}{c}, \qquad b_i = \max(b - p_i, 0).$$

The exact recursion is

$$V(b, c, \varnothing) = 0, \qquad V(b, c, R) = \min_{i \in R} \{w_i(b, c) + V(b_i, c + \Delta_i, R \setminus \{i\})\}. \tag{3.2}$$

The selected item is followed by the policy selected in the resulting state.

Theorem 3.4 (Finite completion optimum)**.** *Recursion* (3.2) *attains the least completion time among all purchase orders in this model, with each purchase made when affordable.*

Proof. Induct on the number of remaining items. The empty state is complete. Every nonempty order has a first item i and a suffix on $R \setminus \{i\}$. The induction hypothesis bounds the suffix below by its recursive value. Taking the minimum over the finite first choices gives a lower bound for every order. Choosing a minimising first item and an attaining suffix achieves that bound. □

There is no advantage to an extra delay in this deterministic model. For a fixed order, move the first delayed purchase to its earliest affordable time. At its former time, the bank is no smaller, since the purchase gain is nonnegative. The remaining purchases can occur at their former times or earlier. Repeating this operation removes every extra delay. This argument does not apply when a delay changes prices, unlocks, buffs, or random-event rewards.

For a purchased subset S, the bank after earliest purchases is $\max(b_0 - \sum_{i \in S} p_i, 0)$. Income is $c_0 + \sum_{i \in S} \Delta_i$. Therefore a subset dynamic program uses $O(n2^n)$ transitions and $O(2^n)$ states. The reference implementation uses rational arithmetic. This theorem does not model repeatable buildings, achievement unlocks, or heavenly resets.

Corollary 3.5 (Payback period)**.** *Cookie Monster ranks purchases by the payback period*

$$\mathrm{PP}_i = \frac{\max(p_i - b, 0)}{c} + \frac{p_i}{\Delta_i}. \tag{3.3}$$

When no candidate is affordable, $\mathrm{PP}_i = E_i - b/c$*. The payback period and the exchange index then give the same ranking.*

Proposition 3.6 (Affordable pairs)**.** *If* $\max(p_A, p_B) \le b < p_A + p_B$*, buying A first is better if and only if* $\Delta_A > \Delta_B$*.*

Proof. Buy A at once. The bank keeps $b - p_A < p_B$, and the remaining $p_A + p_B - b$ cookies arrive at the rate $c + \Delta_A$:

$$T_{AB} = \frac{p_A + p_B - b}{c + \Delta_A}.$$

The expression for T_{BA} is symmetric, with the same numerator. □

With a large bank the exchange index is therefore not exact. Section 8 shows that the effect is small. Over nine scenarios, the bank-independent index or a look-ahead to tier upgrades changes the time to a target by at most 0.35% in continuous play. When purchases occur only once an hour, the change is at most 1.7%.

4 Golden cookies and the bank

4.1 When to hold a "Lucky!" bank

A "Lucky!" golden cookie pays

$$\min(0.15\,b,\ 900\,c)\,g + 13 \tag{4.1}$$

cookies, where g collects the golden-cookie gain multipliers. Below the cap, $b < 6000\,c$, the reward is linear in the bank.

Proposition 4.1 (Marginal value of banked cookies)**.** *Let* "Lucky!" *cookies occur at the rate* λ_L*, and let* $b < 6000\,c$*. A marginal banked cookie earns* $0.15\,g\,\lambda_L$ *cookies per second in expectation. For a frozen income comparison, a divisible investment with yield* $\bar{\Delta}_i/p_i$ *has the larger marginal return if and only if*

$$\frac{p_i}{\bar{\Delta}_i} < \theta, \qquad \theta = \frac{1}{0.15\,g\,\lambda_L}. \tag{4.2}$$

Proof. Below the cap, each "Lucky!" cookie pays $0.15\,g$ per banked cookie. These cookies arrive at the rate λ_L. By linearity of expectation, a banked cookie returns $0.15\,g\,\lambda_L$ per second. The assumed divisible investment returns $\bar{\Delta}_i/p_i$ per second. Compare these two instantaneous marginal returns. □

Actual purchases are indivisible. The inequality alone does not prove an optimal bank policy or a minimum completion time. Below the cap, bank rewards also change as the bank changes.

Without golden cookies or clicks, $\bar{\Delta}_i$ is the CpS gain Δ_i. With both automated, Δ_i undervalues every purchase: golden cookies and clicks multiply a CpS gain about eightfold (Example 4.3). With Δ_i alone, purchases whose payback lies between θ and several times θ would wait, although a purchase is better.

The rate is $\lambda_L = \pi_L/\mathbb{E}[I]$. Here $\pi_L \approx 0.42$ is the probability that a golden cookie is "Lucky!", and I is the interval between golden cookies.

Outcome	Share	Effect
Frenzy	$\pi_F \approx 0.42$	production $\times 7$ for $77\,d\,\mathrm{s}$
"Lucky!"	$\pi_L \approx 0.42$	$\min(0.15\,b,\ 900\,c)\,g$
Building special	$\pi_B \approx 0.10$	CpS $\times(1 + n/10)$ for $30\,d\,\mathrm{s}$; n counts a random type with at least ten copies
Click frenzy	$\pi_C \approx 0.04$	clicks $\times 777$ for $13\,d\,\mathrm{s}$
Cookie storm	$\pi_S \approx 0.0085$	about $105\,d$ drops, each worth one to seven minutes of CpS

Table 2: Golden-cookie outcomes when every golden cookie is clicked at once ($d = 2$ under *Get lucky*).

Remark 4.2 (Golden-cookie interval approximation). Golden cookies spawn between $t_{\min}$ and $t_{\max}$. In each frame of the window, the spawn probability is $((t - t_{\min})/W)^5$, where t is the current frame and the window lasts W frames. Write $I = t_{\min} + x\,(t_{\max} - t_{\min})$. Then

$$\mathbb{E}[x] \approx \Gamma\left(\tfrac{7}{6}\right)\left(\frac{6}{W}\right)^{1/6}. \tag{4.3}$$

Derivation. The probability of surviving to the fraction x of the window is

$$\prod_{k \le xW} (1 - (k/W)^5) \approx \exp\Big(-W \int_0^x u^5\,du\Big) = \exp(-Wx^6/6).$$

With $\int_0^\infty e^{-ax^n}\,dx = a^{-1/n}\,\Gamma(1 + 1/n)$,

$$\mathbb{E}[x] \approx \int_0^1 e^{-Wx^6/6}\,dx \approx \int_0^\infty e^{-Wx^6/6}\,dx = \left(\frac{6}{W}\right)^{1/6}\Gamma\left(\tfrac{7}{6}\right).$$

Example 4.3. Table 1 evaluates the approximation with $\pi_L = 0.42$ and $g = 1$. The base window runs from 5 to 15 minutes. *Lucky day* and *Serendipity* each halve both limits.

Frequency	W (frames)	$\mathbb{E}[x]$	$\mathbb{E}[I]$	θ
Base	18000	0.244	447 s	7.1×10^3 s
Both upgrades	4500	0.308	121 s	1.9×10^3 s

Table 1: Mean golden-cookie interval and the bank threshold θ of (4.2).

In the second case, take building specials on types that average $\bar{n} = 60$ copies (Definition 4.4). Then $1 + \sigma_c \approx 4.5$ and $1 + \sigma_k \approx 6.1$. Three mouse upgrades each add 1% of CpS per click. At $r = 20$ clicks per second, $r\,\delta_i = 0.6\,\Delta_i$, so $\bar{\Delta}_i \approx 8.2\,\Delta_i$. A purchase then waits only if its CpS payback exceeds

$$8.2\,\theta \approx 1.6 \times 10^4 \text{ s},$$

four and a half hours. Early in a run, payback times are seconds to minutes, so spend every cookie. Late in a run, payback times are hours to days, so keep the "Lucky!" bank.

4.2 Values under automation

Cookie Monster's simulator values upgrades by the CpS that they add. Click upgrades and golden-cookie upgrades therefore show a value of zero. Under automation they have a definite value.

Definition 4.4 (Automation values)**.** Let c be CpS and k the income of an auto-clicker at r clicks per second. A purchase adds Δ_i to c and $r\,\delta_i$ to k, where δ_i is its gain in cookies per click, including click power that is a share of CpS. Table 2 lists the golden-cookie outcomes. Per unit of CpS and of click income, they add on average

$$\sigma_c = \frac{(462\,\pi_F + 3\bar{n}\,\pi_B + 25200\,g\,\pi_S)\,d}{\mathbb{E}[I]}, \qquad \sigma_k = \frac{(462\,\pi_F + 3\bar{n}\,\pi_B + 10088\,\pi_C)\,d}{\mathbb{E}[I]}, \tag{4.4}$$

where $\bar{n}$ is the mean count over the building types with at least ten copies. A purchase raises the expected income by

$$\bar{\Delta}_i = (1+\sigma_c)\,\Delta_i + (1+\sigma_k)\,r\,\delta_i. \tag{4.5}$$

Golden cookies bring in

$$G = \sigma_c\,c + \sigma_k\,k + \lambda_L\,g\,\min(0.15\,b,\ 900\,c) \tag{4.6}$$

in all. The expected income, which fills the bank, is $c + k + G$.

Frenzies and building specials multiply CpS, and so also the click power that comes from it. Below the cap, "Lucky!" pays a share of the bank, not of production, so it is absent from (4.5).

The golden-cookie upgrades follow from (4.6). *Lucky day* and *Serendipity* double the frequency, so each is worth G. *Get lucky* doubles every duration and is worth $\sigma_c\,c + \sigma_k\,k$. *Golden goose egg* adds 5% frequency and is worth $0.05\,G$.

With these frozen income estimates, the same algebra gives a pairwise comparison and a marginal bank comparison. Replacing random income by its mean does not preserve expected hitting times in general. Both use the income without current buffs. Otherwise a Frenzy would make every purchase look seven times better for its 77 seconds. The heavenly-upgrade values of Table 3 use the simpler shares

$$\sigma_F = \frac{462\,\pi_F\,d}{\mathbb{E}[I]}, \qquad \sigma = \sigma_F + \frac{\pi_L\,g\,\min(900,\ 0.15\,b/c)}{\mathbb{E}[I]}, \tag{4.7}$$

which count Frenzies and "Lucky!" only.

5 Heavenly chips

Chips arrive over the course of the game. Earning the chips between the budget levels B and $B + dB$ takes the time

$$\frac{q(B)}{S(B)}\,dB,$$

where q is the number of cookies that each chip costs and $S(B)$ is the speed. The cost q does not depend on the purchases. Heavenly upgrade j costs p_j chips and multiplies S by $m_j > 1$. The prestige factor (2.3) is common to all orders, so q absorbs it.

Theorem 5.1 (Ratio rule for multiplicative speed-ups)**.** *Suppose that all listed upgrades must be acquired, the initial chip bank is zero, there are no parent constraints, and $q > 0$ is constant over the budget range. Prices are positive and multipliers exceed one. Then the order of purchases that minimises the total time sorts the items by*

$$\rho_j = \frac{1 - 1/m_j}{p_j} \tag{5.1}$$

in decreasing order. A swap of an adjacent pair strictly improves any order that is not sorted.

Proof. Let Π be the product of the multipliers bought before two adjacent items i and j. A swap leaves every other term unchanged, because after both items the product is $\Pi m_i m_j$ in either order. Let S_0 be the initial speed. The contribution of the pair is

$$\frac{q}{S_0\Pi}\Big(p_i + \frac{p_j}{m_i}\Big) \quad \text{with } i \text{ first}, \qquad \frac{q}{S_0\Pi}\Big(p_j + \frac{p_i}{m_j}\Big) \quad \text{with } j \text{ first}.$$

Hence i first is strictly better if and only if

$$p_i + \frac{p_j}{m_i} < p_j + \frac{p_i}{m_j} \iff p_i\Big(1 - \frac{1}{m_j}\Big) < p_j\Big(1 - \frac{1}{m_i}\Big) \iff \rho_i > \rho_j.$$

An unsorted order contains an adjacent inverted pair, and its swap strictly reduces the total time. There are finitely many orders, so a sorted order is optimal. □

Remark 5.2.

1. For small effects $1 - 1/m \approx \ln m$, so the rule reduces to the log-gain per chip. For large effects the two differ: *Heralds* can be worth $m = 2$.

2. The rule implies *saving up*. A cheaper item with a lower ratio, bought first, delays an item with a higher ratio, and the interchange argument shows that this is worse. Cookie Monster+ therefore spends chips along the sorted order and stops at the first unaffordable item.

3. A fixed ordered block has multiplier $M = \prod_{k=1}^{r} m_k$ and effective duration

$$d = \sum_{k=1}^{r} \frac{p_k}{\prod_{\ell<k} m_\ell}.$$

   Its exchange density is $(1 - 1/M)/d$, not $(1 - 1/M)/\sum p_k$. The latter ignores the speed gained from parents inside the block. For two independent blocks, the compared times are $d_A + d_B/M_A$ and $d_B + d_A/M_B$. This is a block exchange result, not a solution for all precedence graphs. The mod searches ancestor closures and preserves their internal order. Its block choice remains heuristic. Scheduling decompositions provide related ideas [6, 5].

4. The game awards chips at discrete ascensions and often starts with a nonzero chip bank. Chip effort changes with prestige. Those mechanics violate assumptions of the ratio theorem. A saving rule derived from that theorem is therefore a gameplay heuristic.

Table 3 lists the multipliers. They follow the in-game descriptions where the effect on production is direct. Estimates are marked where the effect works through cheaper or newly unlocked store upgrades.

The Unshackled upgrades are not independent. Let u_b and v_t be the unshackle upgrades of building b and tier t. When both are owned, each owned tiered upgrade of b in tier t gives $2 + 0.1\,(20 - b)$ instead of 2 (grandmas: 2.5), a factor g_b. *Unshackled cursors* multiplies the Thousand fingers gain ϕ of a cursor by 25 instead. Let w_b be the share of CpS of building b, T_b its owned tiers, and $U_0 \subseteq U$ the owned and the owned-or-planned unshackle upgrades. Then

$$m(U) = \sum_b w_b \frac{f_b(U)}{f_b(U_0)}, \qquad f_b(U) = \begin{cases} c_b\, g_b^{|\{t \in T_b:\ v_t \in U\}|}, & u_b \in U, \\ 1, & u_b \notin U, \end{cases} \tag{5.2}$$

with $c_{\text{cursor}} = (\beta + 25\phi)/(\beta + \phi)$ for the base output β of a cursor, and $c_b = 1$ otherwise. Each step of a block uses the marginal factor $m(U \cup \{j\})/m(U)$. In a save at prestige 1.43×10^{14},

Heavenly upgrade	Chips	Production multiplier m
Heavenly cookies	3	1.10
Heralds	100	$1 + 0.01 \cdot$ heralds (heralds ≤ 100)
Heavenly luck	77	$1 + 0.05\,\sigma$
Lasting fortune	777	$1 + 0.10\,\sigma_F$
Lucky digit	777	$1.01\,(1 + 0.01\,\sigma_F)$
Lucky number	77,777	$1.01\,(1 + 0.01\,\sigma_F)$
Lucky payout	77,777,777	$1.01\,(1 + 0.01\,\sigma_F)$
Kitten angels	9,000	$1 + 0.1 \cdot$ milk
Halo gloves	55,555	$1 + 0.1\,s_{\text{click}}$
Wrinkly cookies	6,666,666	1.10
Distilled essence of redoubled luck	7,777,777	$1 + 0.01\,\sigma$
Sugar baking	2×10^8	$1 + 0.01 \min(\text{unspent lumps}, 100)$
Sugar crystal cookies	10^9	1.05
Divine discount	99,999	1.01 (estimate)
Divine sales	99,999	1.005 (estimate)
Divine bakeries	399,999	1.03 (estimate)
Synergies Vol. I / Vol. II	222,222 / 2,222,222	1.05 each (estimate)
Cookie boxes, four early / three late	25 / 3.3×10^{11}	1.02 each (estimate)
Persistent memory	500	1.01 (estimate)
Starter kit / Starter kitchen	50 / 5,000	1.005 each (estimate)
Aura gloves	5.6×10^8	$1 + s_{\text{click}}\,a_{10}$
Luminous gloves	5.6×10^{10}	$1 + s_{\text{click}}\big((1 + a_{20})/(1 + a_{10}) - 1\big)$
Cat ladies	9×10^9	$1 + w_G\,(1.29^n - 1)$
Milkhelp	9×10^{11}	$1 + 0.05\,w_G \cdot$ milk
Kitten wages / Chimera	9×10^9 / 4×10^7	1.01 / 1.002 (estimates)
Sucralosia Inutilis	10^9	1.003 (estimate)
Fortune cookies / Pet the dragon	7.8×10^{10} / 10^{11}	1.02 / 1.03 (estimates)
Unshackled upgrades (34)	10^7 to 1.9×10^{16}	Equation (5.2)

Table 3: Heavenly upgrades valued by the ratio rule; s_{click} is the share of income from clicks, $a_L = 0.05 \min(\ell, L)$ for cursor level ℓ, w_G the share of CpS from grandmas and n the number of kitten upgrades owned. Chips are list prices; parents are added when blocks are formed. Cosmetics, switches, offline production, the dragon, wrinklers, seasons and permanent upgrade slots are bought only as parents of valued upgrades, unless Section 7 values them. Sugar craving and its child are not bought: with Sugar craving, autoplay spends each lump on the sugar frenzy.

cursors had $w = 0.34$. *Unshackled cursors* with its parents cost 3.3×10^7 chips and gave $m = 9.27$. All 34 upgrades gave $m = 14.1$.

Two rules use the chips that the order leaves. First, 100 heavenly upgrades earn *All the stars in heaven*. By Proposition 7.1, one achievement multiplied production by about 1.03 in that save. Autoplay buys the cheapest available upgrades still missing when they cost at most 1% of the chips held at ascension. Second, *Lucky payout* is sold only when the decimal prestige level $\lfloor P \rfloor$ contains four 7s. The game earns no cookies during the ascension animation, so the frame that starts the ascension fixes $\lfloor P \rfloor$. Autoplay waits up to 60 seconds for such a frame.

6 When to ascend

6.1 Time rescaling

Lemma 6.1 (Time rescaling)**.** *Assume A1, A3 and A4. If a run at speed* 1 *bakes* $F(x)$ *cookies in the time* x*, then the same run at speed* S *bakes* $F(St)$ *cookies in the time* t*.*

Proof. Let $q(t)$ be the holdings and $r(q)$ the base income. At speed S the bank obeys $\dot{b} = S\,r(q)$ between purchases. By A4, purchase decisions depend only on (b, q). In the time variable $\tilde{t} = St$ the dynamics are $db/d\tilde{t} = r(q)$, with the same decision rule, independent of S. Hence the path $(b, q)(\tilde{t})$ is the same for every S, and the cookies baked, $\int r\,d\tilde{t}$, equal $F(\tilde{t})$. □

Ranking purchases by payback period satisfies A4, since every PP_i scales by $1/S$. Golden cookies satisfy it only approximately, because their spawn clock runs in real time. So does the "Lucky!" reserve, which scales with S.

6.2 Formulation

Run k starts with the all-time cookies A_k and the speed $S_k = S(P(A_k))$. If it lasts T_k, it bakes $C_k = F(S_k T_k)$ cookies by Lemma 6.1, and $A_{k+1} = A_k + C_k$. The player chooses the run lengths to minimise the total time $\sum_k (T_k + \tau)$ needed to reach a distant goal A^*. This is an optimal stopping problem with the Bellman equation [1]

$$V(A) = \min_{A < A' \le A^*} \Big[T(A \to A') + \tau\,\mathbf{1}[A' < A^*] + V(A') \Big], \qquad V(A^*) = 0. \tag{6.1}$$

6.3 Late game: a closed form

In the late game $M(P) \approx 0.01\,hP$, so $S(P) = \kappa P$. Take $\tau = 0$ and assume A5. Let $y = \ln P$. A run whose log-prestige grows by Δ must bake $C = 10^{12} P^3 (e^{3\Delta} - 1)$ cookies. At speed κP this takes

$$T(y, \Delta) = \frac{(C/\alpha)^{1/\beta}}{\kappa P} = K\, e^{\gamma y}\, \varphi(\Delta), \tag{6.2}$$

where

$$\varphi(\Delta) = (e^{3\Delta} - 1)^{1/\beta}, \qquad \gamma = \frac{3}{\beta} - 1, \qquad K = \frac{(10^{12}/\alpha)^{1/\beta}}{\kappa}. \tag{6.3}$$

When $\gamma > 0$ ($\beta < 3$), runs become longer as prestige grows. When $\gamma < 0$, they become shorter.

Theorem 6.2 (Lower bound and optimal step)**.** *Let* $G(\Delta) = (e^{\gamma\Delta} - 1)/\gamma$*, with* $G(\Delta) = \Delta$ *if* $\gamma = 0$*, and let*

$$w = \inf_{\Delta > 0} \frac{K \varphi(\Delta)}{G(\Delta)}.$$

Every ascension schedule from y_0 *to* y^* *satisfies*

$$\sum_k T_k \ge \frac{w}{\gamma}\big(e^{\gamma y^*} - e^{\gamma y_0}\big) \qquad \big(\text{or } w\,(y^* - y_0) \text{ if } \gamma = 0\big), \tag{6.4}$$

with equality if and only if every step Δ_k *minimises* φ/G*. For* $\beta > 1$ *a minimiser* $\Delta^\star$ *exists.*

Proof. **Bound.** For $\gamma \neq 0$ and $\Delta > 0$, $G(\Delta) > 0$ and

$$e^{\gamma y} G(\Delta) = \frac{e^{\gamma(y+\Delta)} - e^{\gamma y}}{\gamma}.$$

By the definition of w, $K\varphi(\Delta) \geq w\,G(\Delta)$ for every $\Delta > 0$. With $y_{k+1} = y_k + \Delta_k$ the sum telescopes:

$$\sum_k T_k = \sum_k K e^{\gamma y_k}\varphi(\Delta_k) \geq w \sum_k e^{\gamma y_k} G(\Delta_k) = \frac{w}{\gamma}\sum_k (e^{\gamma y_{k+1}} - e^{\gamma y_k}) = \frac{w}{\gamma}(e^{\gamma y^*} - e^{\gamma y_0}).$$

The only inequality is an equality exactly when $K\varphi(\Delta_k) = w\,G(\Delta_k)$ for every k. The case $\gamma = 0$ is the same with $G(\Delta) = \Delta$.

Existence. As $\Delta \to 0^+$,

$$\frac{\varphi(\Delta)}{G(\Delta)} \sim 3^{1/\beta}\Delta^{1/\beta - 1} \to \infty,$$

because $\beta > 1$. As $\Delta \to \infty$, $\varphi(\Delta) \sim e^{3\Delta/\beta}$, while $G \sim e^{\gamma\Delta}/\gamma$ for $\gamma > 0$, $G = \Delta$ for $\gamma = 0$, and $G \to -1/\gamma > 0$ for $\gamma < 0$. Since $\gamma < 3/\beta$, the ratio tends to infinity in all three cases. A continuous function on $(0, \infty)$ that tends to infinity at both ends attains its minimum. □

Repeating $\Delta^\star$ achieves the bound only when the goal lies on that sequence of steps. A final partial step can create a gap. Adjusting several steps can improve a finite-goal schedule, so the constant-step rule is not an exact solution for every endpoint. For $\beta \leq 1$ the ratio φ/G has no interior minimum, and resets should be as frequent as the overhead allows. This degenerate case motivates a safeguard in the implementation (Section 6.5).

Corollary 6.3 (Stopping rule)**.** *At an interior minimiser, $\varphi'/\varphi = G'/G$. The current run's length is proportional to $\varphi(\Delta)$, so a step that attains the lower bound satisfies*

$$\frac{d\Delta}{dT} = \frac{1 - e^{-\gamma\Delta}}{\gamma\,T}. \tag{6.5}$$

For a run whose cookies grow like T^β, $\Delta = \frac{1}{3}\ln v$ with $v = (A + C)/A$. A step attaining that bound has a factor $v^\star > 1$ that solves

$$\frac{\beta}{3}\Big(1 - \frac{1}{v}\Big) = \frac{1 - v^{-\gamma/3}}{\gamma}. \tag{6.6}$$

Proof. From $T'(\Delta)/T(\Delta) = \varphi'/\varphi = G'/G$,

$$\frac{d\Delta}{dT} = \frac{G}{G'\,T} = \frac{e^{\gamma\Delta} - 1}{\gamma e^{\gamma\Delta}\,T}.$$

With $C \propto T^\beta$, $dv/dT = \beta(v - 1)/T$, so $d\Delta/dT = \frac{1}{3v}\,dv/dT = \frac{\beta}{3}(1 - 1/v)/T$. Equating the two expressions gives (6.6). □

Remark 6.4 (The marginal value theorem). For $\gamma = 0$ ($\beta = 3$) the rule reads $d\Delta/dT = \Delta/T$: ascend when the marginal rate equals the average rate. This is the marginal value theorem of optimal foraging [2]. Its use at $\beta \neq 3$ ignores that later runs become longer or shorter. It then solves

$$\beta(v - 1) = v \ln v$$

instead of (6.6). The cookie-growth factors differ by 1.6 at $\beta = 2$ and 6.5 at $\beta = 6$ (Table 4).

Remark 6.5 (Overhead). With $\tau > 0$ the problem is no longer self-similar. Corollary 6.3 then holds approximately, with T replaced by $T + \tau$. Dynamic programming solves the stated finite-grid model exactly (Section 6.5). Grid restriction and estimated run curves remain approximation errors.

β	γ	$v^\star$ (6.6)	prestige × per run	naive MVT v	brute force	dynamic program
1.5	1.000	4.24	1.62	2.40	4.42	4.39
2.0	0.500	7.87	1.99	4.92	7.95	7.18
2.5	0.200	12.03	2.29	9.31	11.49	11.50
3.0	0.000	16.80	2.56	16.80	15.95	16.69
4.0	−0.250	28.39	3.05	50.44	31.69	28.95
6.0	−0.500	60.91	3.93	397.38	63.59	60.13

Table 4: Optimal factor $v^\star$ by which each run multiplies all-time cookies in the late game. "Brute force" scans fixed factors to grow prestige a millionfold; the last, partial run causes the scatter. "Dynamic program" is the planner of Section 6.5, averaged over runs that have baked 0.1, 0.5 and 2 times the previous all-time cookies.

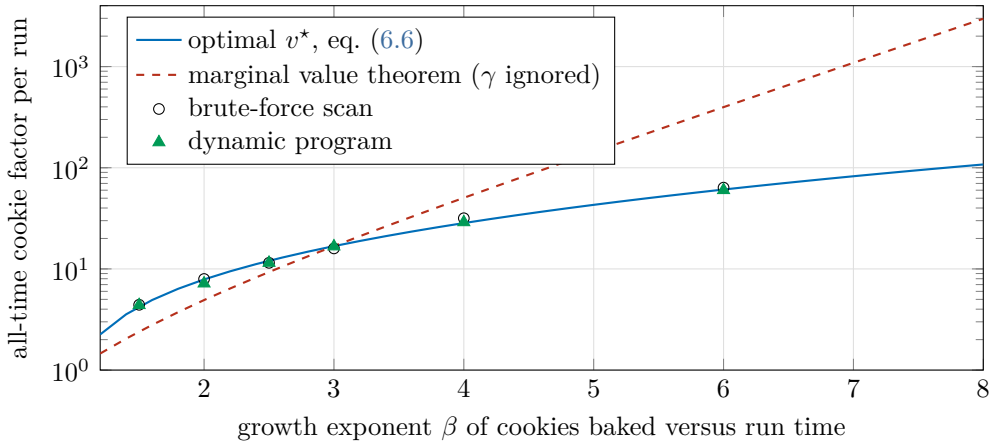


Figure 1: The late-game optimum and the uncorrected marginal value theorem. The two agree only at $\beta = 3$.

6.4 Early game: why early resets fail

Proposition 6.6 (No prestige effect, no reset)**.** *Suppose that every run has the same speed, $S(P) \equiv S$, and that $F(x) = \alpha x^\beta$ with $\beta > 1$. Then for every $\tau \geq 0$, a single run is the fastest way to bake a given number of cookies.*

Proof. Split D cookies into runs of $D_1, \ldots, D_n$. The total time is

$$\sum_k \frac{(D_k/\alpha)^{1/\beta}}{S} + (n-1)\tau.$$

Since $q = 1/\beta \in (0,1)$, the map $x \mapsto x^q$ is strictly subadditive on $[0,\infty)$: $(\sum_k D_k)^q < \sum_k D_k^q$ for $n \geq 2$ positive parts. Hence one run is strictly fastest. □

Early in the game, ascension raises the speed only slightly, so the problem is close to this case and early resets are costly. An earlier version of the advisor applied the late-game rule to a first run of 4.89×10^{12} cookies: prestige 1.70, one level, +1% production. It recommended an ascension. The planner below says to keep baking. It puts the cost of an ascension at that point at 11 to 16 hours of total progress (Table 5).

6.5 The general planner

Speed model. A run that starts at prestige P has the speed

$$S(P) = \Big(\prod_{j \in J(\lfloor P \rfloor)} m_j \Big) (1 + 0.01\, h\, \lfloor P \rfloor). \tag{6.7}$$

Legacy is bought with the first chip. $J(\lfloor P \rfloor)$ is the longest prefix of the order of Theorem 5.1, parents included, that the chips available at P pay for.

Run-time model. The current run has baked C cookies in the time T, and $\beta = cT/C$. A fresh run rebuilds to C convexly and then grows like the current run:

$$t(X) = \begin{cases} T\,(X/C)^{1/\beta_r}, & X \le C, \quad \beta_r = \max(\beta, 1.5), \\ T\,(X/C)^{1/\beta}, & X > C. \end{cases} \tag{6.8}$$

A run at speed S needs $t(X)\, S_{\text{now}}/S$ to bake X cookies (Lemma 6.1). The floor $\beta_r \ge 1.5$ states that every run starts from zero production, even when the current run has stalled.

Proposition 6.7 (Exactness of the dynamic program)**.** *Restrict ascensions to a finite grid of all-time cookie levels $a_0 < a_1 < \cdots < a_n = A^*$, and let $t(i, j)$ be the duration of a run from a_i to a_j. Then*

$$V(i) = \min_{j > i} \Big[t(i, j) + \tau\, \mathbf{1}[j < n] + V(j) \Big], \qquad V(n) = 0, \tag{6.9}$$

computed for $i = n - 1, \ldots, 0$ in $O(n^2)$ time, is the minimum total time from a_i. The minimising j is the optimal next ascension point. For the current run, let r_j be the remaining time to grid level a_j. Its value is

$$\min \Big\{ r_n,\ \min_{j < n} \big[r_j + \tau + V(j) \big] \Big\}.$$

The first term reaches the goal without a reset. It is required even when a reset would increase speed.

Proof. All-time cookies only increase, so schedules are paths in a directed acyclic graph on the grid, with edge weights $t(i, j) + \tau\, \mathbf{1}[j < n]$. The recursion is Bellman's equation (6.1) on this graph. By induction on $n - i$, $V(i)$ is the shortest path length from i to n. □

Four implementation choices keep the planner robust.

- *Horizon.* The goal is the level that the current growth would reach in 10^4 run lengths. A more distant goal would let floating-point error swamp the hour-scale differences of the current decision.
- *End effects.* The target is the median over five goals spaced over one decade of all-time cookies. This reduces sensitivity to the endpoint. The median of policies for different goals need not minimise any one objective.
- *Minimum gain.* An ascension must raise the speed by at least 10%. This guards against a stalled run, for example while nothing is bought. Such a run pushes β toward 1, where Theorem 6.2 favours constant resets.
- *Grid and age.* The grid has 400 log-spaced levels plus every whole prestige level below 400. No plan is made before a run is ten minutes old.

β	ascend now?	optimal target	forecast (growth ±25%) / cost of ascending now
1.5	no	prestige 4 (+14.4% speed)	50.6–129.4 h / 11.4 h
2.0	no	prestige 4 (+14.4% speed)	30.9–68.6 h / 14.2 h
2.5	no	prestige 4 (+14.4% speed)	22.0–45.0 h / 15.2 h
3.0	no	prestige 4 (+14.4% speed)	16.9–33.0 h / 15.8 h
4.0	no	prestige 4 (+14.4% speed)	11.6–21.2 h / 16.3 h

Table 5: The planner on a first run of 4.89×10^{12} cookies in 17.2 hours without Legacy. Prestige 4 pays for Legacy and *Heavenly cookies* together, so the first ascension buys several heavenly upgrades at once.

7 The rest of the game

The preceding results justify restricted comparisons, not a complete-game optimum. The game has further systems, and most of them reset at every ascension. Each system has a rule that uses the long-run income model of Definition 4.4, extended below to wrath cookies and wrinklers. This section states the facts that the rules rest on.

7.1 Achievements

Proposition 7.1 (Value of an achievement)**.** *Each achievement, shadow achievements aside, raises the milk μ by 0.04. Let the kitten upgrades have the factors k_i, and let m be the milk multiplier. CpS carries the factor $\prod_i(1 + k_i m\mu)$, so one more achievement multiplies CpS by*

$$\prod_i \frac{1 + k_i m(\mu + 0.04)}{1 + k_i m\mu} \approx \exp\Big(0.04 \sum_i \frac{k_i m}{1 + k_i m\mu}\Big). \tag{7.1}$$

Proof. The game multiplies CpS by $1 + k_i m\mu$ for each kitten owned. Substitute $\mu + 0.04$ and take the ratio. The approximation is $\ln(1 + x) \approx x$ for each factor. □

On an observed save with eight kittens and $\mu = 9.52$, one achievement adds 2.14% of CpS. This return alone does not prove that every small achievement should be taken at once. Fifty news clicks or a viewport change can be cheap. Holding a golden cookie until its last second delays its reward; repeated wrinkler pokes need recovery time; and a self-gift has a native cooldown. These costs and any lost alternative action must enter the schedule. For the completion objective, each required achievement must eventually be won even when its production return is small.

Proposition 7.2 (What an achievement is worth in time)**.** *Suppose a fixed goal takes $H \geq 0$ more seconds without an action. The action pauses progress for $D \geq 0$ seconds. It then multiplies all future rates by $M > 0$, with unchanged paths. The action improves completion time if and only if*

$$D + \frac{H}{M} < H \quad \Longleftrightarrow \quad D < H\left(1 - \frac{1}{M}\right). \tag{7.2}$$

For $M = 1 + x$, the saved time is $Hx/(1 + x)$, not Hx. For k milk achievements, use the combined factor

$$M = \prod_i \frac{1 + k_i m(\mu + 0.04k)}{1 + k_i m\mu}.$$

Adding k copies of the first achievement's percentage is only a small-gain approximation.

Proof. The action has total completion time $D + H/M$ by time rescaling. Compare it with H and rearrange. Substitute the new milk level for the combined factor. □

Action	Achievements	Cost	Decision
Born again run	Neverclick, Hardcore	a new run of about 4 h: 10^6 cookies in 41 min, 10^9 in 3.8 h	start when $H(1-1/M_2) > 4$ h, with combined factor M_2
Bursting spree	Wrinklesquisher, Moistburster	50 or 200 bursts in one run, $N\tau/n_{\max}$: 15 or 62 min at reduced income	burst when the loss is less than $Hx/(1+x)$ and the run lasts long enough
Cheap spells	spell counts (9, 99)	golden cookies worth about 1.5 h	keep casting the Hand of Fate: the gain is about 0.5 h

Table 6: Actions priced by (7.2). In the spree, each of the $n_{\max} = 12$ wrinkler slots refills after $\tau = 222$ s on average (stage 3 with Unholy bait), and income falls to that of wrath without wrinklers.

A cookie expense converted to $D = C/I$ is a further approximation when income changes. Save age is a chosen horizon estimate, not measured time to completion.

The horizon is unknown. A proper Pareto prior with survival proportional to $1/t$ gives a conditional median remaining time equal to elapsed time. A log-uniform density proportional to $1/t$ is improper without bounds and gives no such conclusion. Autoplay therefore takes H to be the age of the save. On the observed save, after 77 hours, one achievement saves about $77(0.0214)/(1.0214) = 1.61$ hours under this horizon estimate. Table 6 applies (7.2) to three kinds of action.

A Born again run starts from nothing: no prestige, heavenly upgrades, sugar lumps or minigames. A simulation of such a run (buildings only, the 15 permitted clicks spent on the first cursor, golden cookies clicked) gives the times in Table 6. One run earns both achievements. In the run, autoplay clicks the big cookie exactly 15 times, buys only buildings and ascends back into a normal run as soon as both achievements are won.

Cheap spells would roughly double the number of casts, because magic regenerates like $\sqrt{m}$ (Proposition 7.7). They would reach 99 casts in about 22 hours instead of 41.

7.2 Godzamok

Godzamok, one of the Pantheon's gods, rewards the sale of buildings. Each building sold nominally adds $b = 1\%$ to the click power of a ten-second "Devastation" buff (0.5% in the second slot, 0.25% in the third). Sales while the buff runs do not extend its duration. Repeated additions can stop changing its binary64 multiplier when the increment is too small.

Proposition 7.3 (Godzamok round trips)**.** *Assume real arithmetic, geometric prices without free copies or integer rounding, unchanged price modifiers, and no sale hook that changes production. Sell all n copies of a building whose next copy costs p, and buy them back in the same frame. Production does not change, and the round trip costs*

$$(1 - 1.15\,\sigma)\,\frac{p\,(1 - 1.15^{-n})}{0.15}, \qquad \sigma = 0.25, \tag{7.3}$$

which is 0.7125 *times the cost to buy the copies back. The round trip adds bn to the multiplier of the buff, which is worth $b\,n\,K\,t$ cookies. Here K is the click income without the buff and t is the remaining buff time. If U buildings are cycled in each of the $F = 300$ frames of a buff, the click multiplier averages*

$$1 + bU\,\frac{F+1}{2} \approx 1 + 150\,bU. \tag{7.4}$$

Proof. A sale at the amount j refunds σ times the price of copy $j+1$, which is 1.15 times the price of copy j. Summed over the top n copies, the refund is $1.15\,\sigma$ times the buy-back cost, a geometric sum. Click power is linear in the multiplier of the buff, so a sale adds bnK per second for the remaining t seconds. Cycling U buildings per frame makes the multiplier $1 + bUk$ in frame k. Average over $k = 1, \ldots, F$. □

Autoplay cycles building types whose estimated return is more than twice the round-trip cost. Version 2.053.17 projects the bank through every sale credit and purchase debit in native order. It checks affordability at each purchase and uses the resulting bank loss. Summing individually rounded prices is not enough: the bank operations also round. The projection assumes unchanged price modifiers. Version 2.053.21 also computes the immediate increase of a finite multiplier with native operation order. Put $b_{64} = \mathrm{RN}_{64}(b)$ for the stored slot constant. Then

$$\Delta(m, n) = \mathrm{RN}_{64}(\mathrm{RN}_{64}(m + \mathrm{RN}_{64}(nb_{64})) - m)\,.$$

It uses $\Delta(m, n)Kt$ in the return estimate. If an addition rounds away, this click benefit is zero. A cycle can still pass when separate rounded bank operations produce a net gain. The future-average forecast starts each proposed buff at multiplier one; it remains a heuristic. Boundary tests cover all three Pantheon slots and halfway ties. These supplied states do not prove new-save reachability. The mod retains its practical per-frame time and pass limits.

These limits are not restrictions in the zero-input-latency mathematical model. In a native diagnostic state with 101 Mines, Godzamok in the diamond slot, and bank 2^{90}, selling and rebuying 100 Mines leaves the bank and holdings unchanged. All individual payments round away. One thousand such native UI cycles use zero modeled elapsed time and produce a multiplier of 1,001 without advancing its 300-frame duration. Repeating the unit increment has the recurrence

$$m_{n+1} = \mathrm{RN}_{64}(m_n + 1), \qquad m_0 = 1, \qquad m_n = \min(n + 1, 2^{53}),$$

where RN_{64} rounds to nearest binary64 with ties to even. Lean proves this recurrence for its integer rounding projection. It also proves that, at bank 2^{90}, every integer debit at most 2^{36} and refund at most 2^{37} leaves the bank unchanged, including halfway ties. At 2^{53}, another unit is a tie and rounds back to 2^{53}. Larger sale batches can produce larger increments, so this is not a global buff cap. The finite boundary probes do not execute the quadrillions of preceding cycles or establish a legal new-save route. Their significance is that a bounded per-frame search omits admissible mathematical schedules. A full optimality proof must account for those schedules, their native side effects, and their resource prerequisites. A separate finite controller now emits ordinary store inputs for the one-Cursor cent-increment route, with a bank prerequisite of 2^{60} and a deterministic bound of 2^{56} cycles. It maintains its own binary64 cent recurrence and does not read an exact native buff multiplier to choose its stopping point. Six supplied-boundary diagnostics execute 2 181 cycles and check each native input; they do not execute the full bound or assign its endpoint. The controller leaves one Cursor and retains actual sale proceeds. Complete source and public-input correspondence, finite host-resource behavior, later income and restoration costs remain open. It is not enabled in the default candidate or mod.

The mouse upgrades make a click worth 15% of CpS, so at 20 clicks a second $K \approx 3c$ before the buff. On the observed save, about 2,200 buildings per frame kept the multiplier at 3,329 on average. Income rose to about 22,000 times CpS, from a few times CpS before. The game kept 30 frames a second.

The economy model includes the average multiplier (7.4) in click income, so the Pantheon slots Godzamok as soon as a worship swap allows. Each ascension restores three swaps. Once

Stage	Frenzy	“Lucky!”	Special	Click fr.	Storm	Chain	Elder fr.	Clot	Ruin	Cursed
0	.422	.422	.100	.040	.008	.008	–	–	–	–
1	.294	.341	.076	.032	.024	.023	.017	.090	.090	.008
2	.157	.280	.056	.027	.039	.039	.034	.171	.171	.015
3	–	.245	.041	.023	.055	.055	.051	.245	.244	.023

Table 7: Golden-cookie outcome shares by Grandmapocalypse stage, with at least ten buildings and no dragon auras. Building debuffs, 1.8% at stage 3, are omitted.

Outcome	Value	Outcome	Value
Frenzy	$462\,(Wc + k)$	Click frenzy	$10088\,k$
Building special	$3\bar{n}\,(Wc + k)$	Cookie storm	$25200\,c$
Elder frenzy	$3990\,(Wc + k)$	“Lucky!”	$900\,c$
Clot	$-33\,(Wc + k)$	Cookie chain	$1290\,c$
Cursed finger	$2000\,c - 10\,(Wc + k)$	Ruin	$-300\,c$

Table 8: Value of each outcome in cookies, per unit of effect duration, with a full “Lucky!” bank. Buffs count W times because wrinklers digest buffed CpS; payouts count once.

they are spent, the next swap takes 16 hours. When a much better arrangement waits for swaps, autoplay keeps the next sugar lump for the refill that restores all three.

7.3 Wrinklers and wrath cookies

Proposition 7.4 (Wrinkler factor)**.** *With n wrinklers digesting, a share $s = \min(1, n/20)$ of the CpS c withers, and every wrinkler accumulates sc per second. A popped wrinkler returns $r \geq 1.1$ times its contents. Income from CpS is therefore $c\,W(n)$, with*

$$W(n) = 1 - s + s\,n\,r, \qquad W(10) = 6, \qquad W(12) = 8.32 \qquad (r = 1.1). \tag{7.5}$$

Proof. In the game’s update, each digesting wrinkler adds cs per second to its stomach: the whole withered share, not $1/n$ of it. The bank receives $(1 - s)c$. Sum over the n wrinklers and apply the return factor. □

Wrinklers appear only during the Grandmapocalypse. At stage $\ell \in \{1, 2, 3\}$, each golden cookie is a wrath cookie with probability $\ell/3$. Wrath cookies replace Frenzies with clots, which halve production for 66 s, and ruin, which takes $\min(0.05\,b, 600\,c)$. In return they bring Elder frenzies (production $\times 666$ for 6 s), cursed fingers (each click worth 10 s of production for 10 s) and cookie storms: about 105 golden cookies in 7 s, each worth one to seven minutes of CpS. Table 7 gives the outcome shares, simulated from the game’s choice rule.

Proposition 7.5 (Wrath cookies win under automation)**.** *Let the click income be $k = 0.6\,c$, and let building specials be worth 588 s. With a full* “Lucky!” *bank, an average golden cookie is worth 1248 s of CpS at stage 0 and 2141 s at stage 3 without wrinklers. With ten wrinklers, the stage-3 cookie is worth 3236 s. Total income at stage 3 is about 2.8 times that of stage 0.*

Proof. Multiply the shares of Table 7 by the values of Table 8 and sum. □

Autoplay therefore researches up to the Elder Pact. It buys the research that the game guards with a confirmation prompt. Godzamok changes this verdict.

Proposition 7.6 (Calm wins once clicks dominate)**.** *With the Elder Pact, a pledge costs at most* $8^{14} \approx 4.4 \times 10^{12}$ *cookies and returns by itself when it runs out. The stage is thus a free choice between calm (stage 0, no wrinklers) and wrath (stage 3,* $W(n_{\max})$*). Take* $d = 2$*, a mean of* $\bar{n} = 200$ *copies per building type,* $\mathbb{E}[I] = 121\,s$ *and twelve wrinklers. Golden cookies then multiply click income by*

$$1 + \frac{(595 + 0.30\,\bar{n})\,d}{\mathbb{E}[I]} = 11.8 \text{ (calm)}, \qquad 1 + \frac{(428 + 0.12\,\bar{n})\,d}{\mathbb{E}[I]} = 8.5 \text{ (wrath)}, \tag{7.6}$$

and CpS income by 8.7 *(calm) and* 62.8 *(wrath). Calm earns more if and only if*

$$\frac{k}{c} > \frac{62.8 - 8.7}{11.8 - 8.5} \approx 16. \tag{7.7}$$

Proof. For clicks, Frenzies, building specials and Click frenzies bring $(595 + 0.30\,\bar{n})\,d$ buff-seconds per golden cookie in calm. In wrath, Elder frenzies and Click frenzies, less clots, cursed fingers and debuffs, bring $(428 + 0.12\,\bar{n})\,d$. The shares are those of Table 7. Divide by $\mathbb{E}[I]$ and add one. For CpS, the same buffs count W times, and storms and cursed fingers pay directly. Compare $8.7\,c + 11.8\,k$ with $62.8\,c + 8.5\,k$. □

Without Godzamok, clicks bring about $3c$ and wrath wins. With him, clicks bring thousands of times c, and calm earns about 40% more. Autoplay compares the two incomes with the economy model and renews the pledge whenever calm wins. It lets the wrath reach stage 3 once, for the Grandmapocalypse achievement. It then runs the elder achievements below and revokes the covenant that they leave, which costs 5% of CpS.

The bank threshold also changes with the stage, because ruin takes 5% of the bank. Per banked cookie,

$$\frac{1}{\theta} = \frac{0.15\,\pi_L g + 0.215\,\pi_{\text{chain}}\,g - 0.05\,\pi_{\text{ruin}}}{\mathbb{E}[I]}, \tag{7.8}$$

so θ rises from about 1.9×10^3 s at stage 0 to 3.3×10^3 s at stage 3.

When to pop. Wrinklers lose nothing by waiting, but their contents earn nothing until popped. Pop a wrinkler that holds X cookies now, T seconds before the ascension. Its cookies then grow to $X\,e^{\rho T}$ by the end, where $\rho = (\beta - 1)/t$ is the income growth rate of the run. Its slot stays empty for the respawn time τ, and meanwhile the other wrinklers each digest a twentieth less. The loss is

$$L = \big(0.05\,(2n - 1)\,r - 0.05\big)\,c\,\tau, \tag{7.9}$$

and it compounds to $L\,e^{\rho T}$. Autoplay pops the fattest normal wrinkler when

$$X > \frac{L\,e^{\rho T}}{e^{\rho T} - 1}. \tag{7.10}$$

Elder achievements. With the Elder Pact owned, the pledge returns as soon as a signed covenant is revoked. Pledge, covenant and revoke, repeated five times, earn all three elder achievements in seconds for a negligible amount of cookies, instead of two and a half hours of peace.

7.4 Minigames

Proposition 7.7 (Fixed-capacity casting frequency)**.** *Fix a magic cap* $M > 0$*, a spell cost* $0 < C \le M$*, and a regeneration constant* $\kappa > 0$*. Assume continuous regeneration at rate* $\kappa\sqrt{m}$*, instantaneous casts and no other changes to magic. At zero magic, use the refill solution* $\sqrt{m(t)} = \kappa t/2$ *until the cap. Casting whenever magic reaches* M *maximizes the asymptotic number of casts per unit time.*

Proof. A refill from a to $a + C$ takes

$$\int_a^{a+C} \frac{dm}{\kappa\sqrt{m}} = \frac{2}{\kappa}(\sqrt{a+C} - \sqrt{a}),$$

which decreases in a. Define the potential $H(m) = 2\sqrt{m}/\kappa$ and $D = H(M) - H(M - C) > 0$. During regeneration, H increases at rate one; at the cap it stays constant. Each cast from any legal magic value decreases H by at least D. For every schedule with n casts by time T, telescoping these changes gives

$$nD \le T + H(m_0) - H(m_T) \le T + H(m_0).$$

Thus the limiting upper cast frequency is at most $1/D$, including for nonperiodic schedules. Full-capacity casting attains this frequency, with one cast every D time units after the initial refill. □

With $M \approx 72$ and Force the Hand of Fate at $C \approx 53$, a cast at full magic gives one cast every 23 minutes instead of every 40. These figures use the continuous approximation. The native game regenerates in discrete updates, applies binary64 rounding and a positive minimum rate, and changes spell prices when the magic cap changes. Tower purchases, sales, sugar refills and ordinary ascensions also change the available schedule. The proposition does not minimize time to a fixed spell-count goal or to full-game completion. For example, if only one cast remains and current magic already covers its cost, waiting for the cap delays that goal.

The revised mod uses remaining magic immediately before an already-planned ordinary ascension. After cash-in, loans and the Lucky prestige wait, it casts the cheaper affordable spell from Haggler's Charm and Diminish Ineptitude until the 999-cast achievement is won or neither is affordable. Both native outcomes increase the spell count without changing cookies or lumps, and Reset clears their temporary buffs. The setting, challenge and transition guards remain active. This adds no reset and no regeneration wait. Eleven regression cases include both outcomes of both source spells and the final ascension gates. This is a count-progress heuristic, not a full-game dominance or optimality theorem; seeded spell state and later observations still depend on the cast history.

A separate experimental completion phase uses ordinary resets to obtain native magic at a fixed cap. With Wizard tower level 1, Reset sets the zero-tower cap and current magic to 5. Buying one tower retains that cap, and Conjure Baked Goods costs 4. Either native outcome increments the persistent spell counter. The candidate casts once, then resets on the same schedule after either outcome. It does not cancel a child timer or search for a favorable seed. A continuous source audit from a declared 10^{57}-cookie, one-lump entry completes 999 casts in 5 335 760 milliseconds: 859 successes and 140 backfires, with no later resource injection or extra lump spending. These zero-profit resets leave the profitable-reset counter unchanged. The controller therefore requires the separate 100-reset milestone and growth goals first, and finishes its last reincarnation before final Garden and market collection. This is a conditional source witness, not a proved new-save route or a uniform expected-time bound.

Pantheon. Three slots take three of eleven gods. Autoplay evaluates every ordered triple of the useful gods with the game's CpS formula and the income model. The golden-cookie penalties of Mokalsium and Jeremy enter through the interval. Autoplay moves gods only when income rises by more than 1% and worship swaps are available.

Garden. Most new plant kinds appear by mutation in empty tiles next to the required parents. Some recipes accept immature parents. Meddleweed can appear without parents; uprooting

Loan	m_i	t_i	m'_i	t'_i	f_i
1	1.5	2 h	0.25	4 h	20%
2	2	40 s	0.1	40 min	40%
3	1.2	48 h	0.8	120 h	50%

Table 9: Stock Market loans: the boost, its length, the interest that follows, its length and the down payment.

old meddleweed can release brown mold or crumbspore. Harvesting a mature plant normally unlocks its seed. Pebbles can also unlock a seed when the plant dies of old age.

Lemma 7.8 (Checkerboard breeding)**.** *On a rectangular plot of at least* 2×2 *tiles, plant the parent kind* A *on the tiles with* $x + y$ *even and* x *even, and the kind* B *on the tiles with* $x + y$ *even and* x *odd. Leave the tiles with* $x + y$ *odd empty. Then every empty tile touches both kinds (two parents of kind* A *when* $A = B$*), so every empty tile can mutate into a seed bred from* A *and* B*.*

Proof. An empty tile (x, y) has $x + y$ odd. Its horizontal neighbours $(x \pm 1, y)$ are parent tiles with the parity of $x + 1$, and its vertical neighbours $(x, y \pm 1)$ are parent tiles with the parity of x. In a plot at least two tiles wide and tall, at least one tile of each pair lies inside the plot. □

Autoplay breeds along the chains whose parents can be planted: Thumbcorn from wheat, then Cronerice, Gildmillet, Clover, Shimmerlily, Elderwort, Golden clover and Bakeberry. It uproots whatever else sprouts in the empty tiles. It lets each new kind grow up and picks it when mature, which unlocks its seed. Wood chips (three mutation rolls per tick, from 300 farms), or else fertilizer (a tick every three minutes instead of five), speed this up.

Thumbcorn, at +2% click power, is worth 2% of nearly all income once Godzamok multiplies clicks. Golden clover, at +3% golden-cookie frequency, is worth 3% of the golden-cookie income. The plot is cleared at every ascension, so breeding advances only in runs long enough for the parents to mature.

With nothing left to breed, autoplay plants a seed j on clay (effects $\times 1.25$). Let the seed add v_j to long-run income over the mature lifetime T_j, for the price p_j. It is planted when

$$v_j T_j > p_j \qquad \text{and} \qquad \frac{p_j}{v_j} < \text{payback of the next purchase.} \tag{7.11}$$

Plants are harvested while mature, just before the growth tick that could kill them, which counts toward the harvest achievements. One tile stays empty for mutations.

Stock Market. Prices revert toward the resting value $R_i = 10 + 10\,i + (\ell_{\text{bank}} - 1)$. A purchase costs at most 20% extra and a sale costs nothing. A purchase below $0.6\,R_i$ and a sale at R_i therefore return at least

$$\frac{1}{0.6 \cdot 1.2} - 1 = \frac{7}{18} \approx 38.9\% \tag{7.12}$$

per round trip completed at those prices. This conditional return does not bound waiting time or opportunity cost.

Offices and loans. Office upgrades sacrifice 100 to 700 cursors and need cursor levels 2, 4, 8, 10 and 12. They open the loans of Table 9. Loan i multiplies CpS by m_i for the time t_i, and then by $m'_i < 1$ for t'_i, against a down payment of a share f_i of the bank.

A run that ends before t_i never pays the interest, and the ascension also earns Debt evasion. Autoplay takes a loan once the planner's slow estimate of the time left is at most $0.9\,t_i$. It ends the run early when a loan is about to run out and

$$\text{cost of ascending now} < (1 - m_i')\,\min(t_i',\ \text{time left}). \tag{7.13}$$

Right before an ascension, the bank is about to be lost, so autoplay also takes any open loan then.

7.5 The dragon, seasons, sugar lumps and permanent slots

Dragon. Sacrificing n copies of a building whose next copy costs p costs

$$p\,\frac{1 - 1.15^{-n}}{0.15} \approx 6.67\,p \qquad (n = 100) \tag{7.14}$$

to buy back. The dragon starts over every run. Radiant Appetite doubles production and needs the sacrifice of 100 copies of each of the first 14 buildings. Training toward it goes ahead once all those buildings are at hand and buying them back costs at most an hour of income. The same rule holds for Dragon Guts, which adds two wrinklers that digest and return 20% more, so that W reaches about 11.7. Auras are chosen by evaluation, and a change costs one top building.

Seasons. Christmas brings the 14 levels of Santa, which cost

$$\sum_{j=1}^{14} j^j \approx 1.2 \times 10^{16} \tag{7.15}$$

cookies in total, with a gift at each level (Santa's legacy gives +3% per level). It also brings reindeer, each worth one to two minutes of CpS. Valentine's day brings seven heart biscuits, +2% each, and Easter brings eggs. Each run visits Christmas until Santa is complete, Valentine's day once the hearts cost a minute of production, and Easter until seven eggs while that achievement is missing. Each switch costs 1.5 times the one before, so the run waits in Christmas for cheap hearts and stays in Valentine's day until the last heart. It otherwise stays in Christmas. The auto-buyer keeps up to a minute of production for the next seasonal action. A purchase that pays back within a minute goes first, because it delays the action by less than a minute.

Sugar lumps. Lumps go first to the minigames and then to the Sugar frenzy of each run, once Sugar craving is owned: production ×3 for an hour, two hours of income for one lump. Next come cursor levels until the first loan opens, 11 lumps counting the bank's own level, for +50% CpS in the last two hours of every run. The rest go to building levels. A level of building b adds 1% of its CpS, a share

$$\frac{0.01\,s_b}{1 + 0.01\,\ell_b} \tag{7.16}$$

of CpS, for $\ell_b + 1$ lumps, plus the value (7.1) at level 10. Levels are bought by gain per lump, saving up for the best one as in Remark 5.2. The mature but unripe Hand-picked attempt is taken once the minigames are open. Its nominal failure probability is one half under a fresh uniform draw. Section 1.4 explains why this is not an unconditional model of every legal interaction.

Speed, target	play	payback period	exchange index	with look-ahead
$S = 1$, 10^{15}	continuous	68.73 h	0.00%	+0.03%
	every 10 min	70.33 h	−0.17%	−0.09%
	every 60 min	77.39 h	−0.06%	−0.08%
$S = 20$, 10^{20}	continuous	36.58 h	0.00%	+0.09%
	every 10 min	38.62 h	−0.02%	+0.01%
	every 60 min	46.65 h	−0.97%	−1.68%
$S = 200$, 10^{25}	continuous	627.81 h	0.00%	+0.35%
	every 10 min	629.95 h	0.00%	+0.35%
	every 60 min	638.06 h	−0.12%	+0.19%

Table 10: Time to bake a target number of cookies. The last two columns give the speed-up relative to the payback period; negative means slower.

Permanent slots. Milk carries over, so a kitten kept in a slot multiplies CpS by $1+km\mu$ from the first second of each run, and a cookie upgrade by 1 plus its power. The slots take last run's upgrades in decreasing order of the logarithm of that multiplier. These systems also enter the heavenly-upgrade values, as estimates: How to bake your dragon 1.8, Season switcher 1.3, permanent slots 1.12 down to 1.03, Elder spice 1.25, Unholy bait 1.05, Sacrilegious corruption 1.04 and Wrapping paper 1.02.

8 Historical numerical reports

Audit status. The tables below were present before this audit. The repository does not contain the simulator or raw traces needed to reproduce them. They have not been rerun after the corrections. They are retained for provenance and cannot certify the revised policy. Other historical save observations also lack reproducible traces.

Simulator. The simulator uses the game's formulas for the base price and production of all 21 buildings. It adds tiered upgrades that double a building at 1, 5, 25, 50, 100, ... copies, global +10% upgrades whose prices grow twentyfold, a milk proxy $1 + n/K$ for n buildings, and clicks at 5 cookies per second. Purchases are greedy by the exchange index. Ascension rules are compared by the prestige reached after a fixed play time, as a percentage of the best rule for that horizon, averaged over horizons of 60 to 960 days.

Purchase order. Table 10 compares the payback period with the bank-independent index of Theorem 3.1, and with a look-ahead that values a building together with the tier upgrade that it unlocks. The differences are negligible in continuous play. In infrequent play, where large banks build up, the payback period is slightly better.

Ascension timing. Table 11 compares the dynamic-programming planner, the stopping rule of Corollary 6.3 with β measured live, the uncorrected marginal value theorem, and fixed rules that ascend when all-time cookies have grown v-fold, for v from 1.5 to 500.

Both the planner and the stopping rule beat the uncorrected marginal value theorem in every scenario, and the hindsight-tuned fixed rule on average. In these late-game scenarios the simpler stopping rule is slightly better: 98.6% against 97.6%.

The gap comes from extrapolation. The planner projects the current growth exponent forward, but simulated runs keep decelerating, so a continuation looks slightly cheaper than it is. In the first scenario, the planner's first two runs last 1033 and 917 hours, against 879 and 833 for the stopping rule. This costs about five points. A run-curve model that anticipates

Scenario	planner (DP)	stopping rule	naive MVT	best fixed v	worst fixed v
$K = 2500$, $A_0 = 10^{15}$	93.5	98.2	93.9	79.3 ($v = 60$)	13.4 ($v = 1.5$)
$K = 2500$, $A_0 = 10^{24}$	98.5	98.7	93.9	98.7 ($v = 8$)	79.1 ($v = 1.5$)
$K = 500$, $A_0 = 10^{15}$	98.8	98.0	90.8	84.5 ($v = 12$)	20.5 ($v = 1.5$)
$K = 500$, $A_0 = 10^{24}$	98.2	98.7	94.0	97.9 ($v = 12$)	68.2 ($v = 500$)
$K = 100$, $A_0 = 10^{15}$	98.1	98.9	91.1	92.8 ($v = 12$)	34.1 ($v = 1.5$)
$K = 100$, $A_0 = 10^{24}$	98.3	99.3	91.6	98.9 ($v = 8$)	45.6 ($v = 500$)
Mean	97.6	98.6	92.6	92.0	–

Table 11: Prestige reached as a percentage of the best rule at each horizon, averaged over five horizons from 60 to 960 days. K is the milk parameter and A_0 the all-time cookies at the start. The "best fixed" rule is chosen in hindsight per scenario; no single v is best everywhere.

Rule	30 days	120 days	prestige at each ascension (120 days)
Planner (DP)	**658.3**	**18,512.5**	4, 206, 815, 1759, 3789, 7321, 13092, 18512
$P_{\min} = 150$, $v = 20$ (best fixed)	421.2	17,939.4	150, 407, 1105, 3000, 8143, 17939
$P_{\min} = 150$, $v = 8$	359.7	17,686.5	150, 300, 600, …, 17686
$P_{\min} = 50$, $v = 20$	368.8	17,137.6	50, 135, 368, 1000, …
$P_{\min} = 4$, $v = 3$	68.1	6,812.0	4, 5, 8, 12, 17, …

Table 12: Final prestige from a fresh game. Twelve fixed rules were tested; the best and three others are shown.

deceleration is the natural fix. The stopping rule, however, assumes $S \propto P$. The planner can operate from a fresh game, but its predictions remain model estimates.

From a fresh game. Table 12 starts with no prestige, so Legacy, *Heavenly cookies* and the prestige power all enter through the speed model. Fixed rules ascend once prestige is at least $P_{\min}$ and all-time cookies have grown v-fold.

9 Implementation

The rules run in Cookie Monster+ [3], a fork of the Cookie Monster mod [4] for the Steam version of the game (2.053). Every second the mod records the ascension inputs. Every five seconds it solves the dynamic program again. Earlier latency claims lacked reproducible traces and are not used as validation here. With automation on, the mod also does the following.

- It clicks the big cookie 20 times per second. The game ignores clicks less than 20 ms apart. The browser throttles timers in a hidden window, and the game makes up at most five seconds per run of its loop. While the window is hidden, a worker's timer therefore runs the missed logic frames and clicks, with the game's own catch-up.
- It clicks every golden cookie and reindeer as soon as it appears.
- In every frame with Godzamok slotted, it sells and buys back the building types of Proposition 7.3 within three milliseconds. The counts are restored before the frame ends, so only click power is computed again.
- In every frame, it buys items in order of payback period, computed with the automation values of Section 4 and the "Lucky!" reserve of Proposition 4.1. It skips switches, sugar-lump costs and the Chocolate egg. A frame uses the values from its start for at most

four milliseconds, and it buys consecutive copies of a building in one call. More copies of a building only raise the values of other purchases, so these values never overrate a purchase. The frame stops after an upgrade, and before a second purchase that needs a reserve, because the "Lucky!" bank grows with CpS. Cookie Monster recomputes the values on the next frame. Earlier versions bought one item every ten frames, which left the start of each run far behind its bank.

- On nine of every ten frames, it also runs one of the systems of Section 7: the Grandmapocalypse (stage choice, sprees and wrinklers), the dragon, seasons, sugar lumps, the Pantheon, Grimoire, Garden and Stock Market, and the achievements that only take an action.

- Once the planner has said "ascend now" for ten seconds, it pops wrinklers, waits for their payouts while other spending is paused, buys the Chocolate egg and ascends. The payout forecast includes Dragon Guts. Version 2.053.15 checks pending payouts instead of adding a fixed two-second wait. It sends the native Escape input after its own ascent request; this ends the five-second ascent animation on the next frame. It also removes the fixed 1.5-second wait on the heavenly screen. It spends chips along the order of Theorem 5.1, parents first and saving up for the next item. It fills the permanent slots and reincarnates, into a Born again run when Proposition 7.2 says that the challenge pays. It ascends early when a loan is about to reach its interest.

Version 2.053.20 retains the native click-sound callback during muted auto-clicks. Earlier versions replaced that callback and skipped its random draw and sound-index update. The new implementation temporarily uses zero sound volume and restores volume and pointer fields after the native click, including an exceptional return. Paired source tests retain the same economic state, sound index and subsequent random output under both free and seeded generators. They also check the native rejection of a repeated same-time click.

The regression suite checks the no-reset alternative, combined achievement gains, discounted block costs, affordable chip prefixes, safe buybacks, and payout ordering. An integration test runs the actual ascent controller against the pinned engine. It checks that wrinkler payouts precede the egg purchase and that the native Escape event advances the ascent. Tests of a model do not prove equivalence with the game. Section 10 distinguishes kernel-checked statements from JavaScript tests.

10 Formal verification and audit scope

The `lean/` project contains the machine-checked statements. Its README gives the pinned Lean version, build command, theorem names, and exact hypotheses. The reference completion solver is `tools/completion-model.mjs`. It uses exact rational arithmetic for finite, fixed purchases. The regression suite includes Example 3.3 and exhaustive small-instance comparisons.

The main deterministic result is `complete_global_optimal`. It proves attainment and comparison against every affordable rational-time completion schedule for the fixed purchase list. Nonnegative gains and positive initial income are required. The result includes arbitrary extra waits. The theorem `greedy_counterexample` checks the exact counterexample, and `achievement_break_even` checks the corrected delay inequality.

The stochastic result `Stochastic.certified_optimal` uses finite probability laws and policies with a finite depth bound. Each action has a nonnegative duration. Probabilities are nonnegative rationals that sum to one. Every policy leaf must satisfy the supplied completion

predicate. A value function that bounds every legal action from below, with equality along an attaining policy, proves optimal expected time among those policies. A three-state example checks an expected optimum of 5/2 seconds. No unfinished leaf receives a free terminal cost.

The generated Lean completion predicate uses the Steam catalogue and the explicit terminal profile from Section 1.3. Checked statements show that missing a required item prevents completion and that shadow flags do not change completion. The exact partition of the 622 normal achievements also proves that completion requires at least two distinct won achievements from the 24 individual sugar exceptions. A synthetic terminal witness checks consistency of the predicate; it does not establish reachability from a new save. Generation checks detect stale identifiers. They do not prove equivalence between the JavaScript implementation and the Lean function.

The Garden trace theorem proves a further necessary condition. From an initial state without Juicy queenbeet or Seedless to nay, full completion requires at least two acquisitions of the Juicy queenbeet seed and two distinct states with a complete seed log. Its transition premise requires the first Seedless award to follow a complete log and to reset that seed. The trace checker tests this premise on supplied adjacent observations. Native tests separately exercise sacrifice, mature harvest, and pebbles unlocking a seed on old-age death. Consequently, the theorem does not incorrectly require two manual harvests. It remains conditional on the trace contract; no proof covers all executions of the JavaScript source.

The age audit gives a finite candidate state component. Native growth adds an integer from `randomFloor`. Mortal ages remain integers below 100 between updates. The two immortal plants have caps 91 and 76, reduced to 86.5 and 72.25 after Seedless to nay. Lean checks those caps with the actual binary64 operations. For the integer-growth projection, it proves closure of the 102-value domain consisting of integers 0–99 and the two fractional caps. A projection that keeps mortal and non-mature immortal ages, but groups mature immortal ages, is preserved by equal growth outcomes and by current-save truncation. The full native growth bound, helper execution and heap correspondence remain proof obligations.

This age projection does not identify complete public observations. The native Elderwort growth indicator is at 221 pixels for age 86.5 and at 220 pixels after save truncation to 86. Lean checks that difference; 3 612 native source-fragment fixtures check the growth and clamp operations with supplied integer growth outcomes. A full-game reduction must retain the extra observation or prove that it cannot improve economic control. A finite list of age values alone does not prove an optimal strategy exists.

The unbounded stochastic layer compares the suprema of expected accumulated costs of finite truncations. Its verification rule requires local Bellman inequalities, equality for the proposed policy, and a vanishing expected residual potential for finite-cost alternatives. This removes the finite-depth restriction on that cost theorem. It is still conditional on the supplied transition system and certificate. It also needs a separate connection to completion time: an infinite sequence of zero-duration actions can have zero accumulated cost while never reaching the target. The formal examples expose this issue rather than assign zero completion time to an unsuccessful path.

The separate `HittingTime` construction defines completion time directly from execution prefixes. For a cap H and prefix depth n, a completed path contributes $\min(T, H)$, and an unfinished prefix contributes H. Write the resulting exact finite expectation as $C_{H,n}$. Depth increases can only decrease this quantity. The completion-time value is represented by the rational lower cut of

$$\sup_{H\in\mathbb{N}} \inf_{n\in\mathbb{N}} C_{H,n}.$$

The order of these limits matters. Taking a supremum over unfinished depth penalties would give a false infinite value before a successful zero-time retry. The checked construction gives infinite time to a permanent zero-time stall and zero expected additional time to the known-missing pet retry. Its finite transition programs can consume an adaptive number of independent 53-bit draws. Deterministic seeded calculations remain part of hidden state. A policy receives its public observation and action history. Random startup is averaged before that policy acts, and legality is required only on reachable unfinished histories. These definitions do not supply the game's transition program or its global optimality certificate.

The randomized-controller extension carries private memory and permits fresh independent random choices at every decision. It places no bound on the total number of decisions or private draws. A capped full-state function V_H is a lower certificate if it is at most H, is at most the capped elapsed time in every complete state, and satisfies $V_H(s) \leq \mathbb{E}[V_H(S') \mid s, a]$ for every legal action in an unfinished state. Lean proves that this certificate bounds every finite-prefix expectation of each controller, then the corresponding expected-time lower cut. Deterministic public controllers retain exactly their previous values, and a permanent zero-time stall still has infinite expectation. Each individual decision sampler and native transition must terminate; unrestricted internal divergence remains outside this extension. No such certificate has yet been established for the full game's transitions. A lower bound may use a larger competitor class, but an attaining policy must separately satisfy the ban on timing chosen to change seeded outcomes.

The theorem `conditional_hand_picked_bound` uses a rational survival-tail certificate at integer times. It verifies the finite-tail argument of Section 1.4 and the conditional bound of 128 872 038 milliseconds. The concrete `SugarClock` transition system covers unripe and ripe harvests, failure resets, automatic harvests, initial activation, ordinary lump gains and spending, seeded type changes, offline whole-interval advances, and resets without rollback. It proves minimum attempt spacing and the inclusion of the no-success event in the no-completion event. The theorem `sugar_run_expected_lower_bound` uses these results to derive the clock bridge. Eleven native transition projections are separately checked by the Lean kernel, and the audit accounts for all seven source assignments to the lump timer. This finite test does not prove that every engine execution refines the transition system. Conditional failure, projection of actual runs, and finite-tail expectation soundness remain premises. Positive probability of noncompletion gives infinite hitting-time expectation. This is a necessary bound, not an optimality certificate.

The executable game adapter loads eight hash-pinned files, including the installed engine and all four minigames. It uses isolated storage, an explicit clock, a reproducible random stream, and a Steam interface fixture. Tests cover native click throttling, purchases, passive income, minigame loading, catalogue identity, and action replay. Startup now follows the inline page scripts, native load handler, language loader, launch, and filler-image callback. This restores a hidden top-bar draw and the console-greeting draw before seed creation. The default fixture fixes 100 heralds, English, zero asset latency, and a UTC start time. Its random tape is a reproducibility device, not a proof of a probability law. The normalized observation is not the full engine state.

A separate Lean source interpreter now makes JavaScript evaluation explicit. Closures contain syntax and lexical-environment indices; heaps, property descriptors, exception completions, and suspended callbacks are data. Its typed decoder accepts 153 252 syntax nodes in 40 units from the pinned scripts and complete inline handlers. A decoded program is not yet a verified translation of its source bytes. Missing host operations remain unresolved model states. A runner fuel limit leaves an execution running; it does not turn model incompleteness into a game failure or an infinite completion time.

The interpreter represents ordinary numbers by their 64 bits and uses Lean's logical binary64 arithmetic. Decimal conversion and the required V8 mathematical functions have explicit implementations. Their 72 384 finite comparisons use the installed Electron 11.5.0 engine, with V8 8.7.220.31. Unicode 13 tables supply UTF-16 casing and regular-expression character properties. Tests compare 11 456 strings and all 1 114 112 Unicode code points for the three recorded properties. A source regular-expression matcher passes 947 native comparisons. String matching, searching, splitting, and replacement preserve the tested conversion, callback, and `lastIndex` order. The combined evaluator suite passes 291 cases, including 105 executions of actual game helper functions.

The arithmetic correspondence now covers the interpreter's actual residual-bit shifts, ties-to-even rounding, normalization, and packing. For every integer $0 \le n < 2^{1023}$, the packed result represents the integer-rounding projection exactly. This includes zero and small integers whose canonical exponent is negative. Separate theorems connect the interpreter's addition and affordable subtraction to this projection. These are universal kernel proofs for the stated operands, not finite numerical comparisons.

The extracted `Game.Earn` and `Game.Spend` bodies now execute in Lean for arbitrary operand bit patterns. The proofs check their assignment order and normal completion; separate call proofs include argument binding, the undefined return value, and restoration of the caller. Body proofs retain arbitrary extra objects, environments, and non-counter fields. The game object can occur at any heap index, with either counter-field order and arbitrary fields before, between, and after the counters. The whole-machine equality preserves unrelated state. The entry still supplies explicit declarative bindings. A further body proof uses the builtin driver with a local parameter and an outer Window data binding for `Game`. Correspondence of these explicit entry layouts to the initialized game's environment chain, data-field invariants, and actual calls remains to be proved.

For fractional inputs, let $N/2^d$ be the exact aligned sum, with $N \ge 2^{52+d}$. For $53 \le B \le 1021$, Lean proves

$$N \le 2^{B+d} + 2^{B+d-53} \quad \Longrightarrow \quad \mathrm{RN}_{64}(N/2^d) \le 2^B.$$

The result is finite and nonnegative. The proof includes the halfway case and does not require 2^d to divide N. Thus a power-of-two counter barrier can apply to fractional income. Each source callback must still establish the corresponding input bounds.

Array sorting executes an explicit translation of the pinned V8 TimSort, including the actual source comparator. Comparisons cover 2 505 arrays and 1 975 984 comparator calls; 32 Lean source cases test sparse arrays, mutation, getters, errors, and merge paths. URI, legacy escape, and base64 conversion pass 263 326 native comparisons. These finite checks are reproducible compatibility evidence, not universal source-refinement theorems.

For every allowed free-draw index $0 \le k < 2^{53}$, Lean proves that the direct bit encoder returns a finite nonnegative binary64 number with exact rational value $k/2^{53}$. Distinct indices give distinct encodings. A further kernel theorem proves that the actual binary64 multiply by two and floor select each of the two indices with exact probability 1/2. The intact `choose([0,1])` expression now has a source-call proof. It resolves the native global declaration, creates the two-element array and mapped arguments object, and reaches the free-draw request in 43 interpreter steps. After that draw, 18 steps return the selected native number to the original caller. All unrelated state and both allocations are retained. The composed unripe-harvest branch reaches that request in 75 source steps. After the draw, 31 steps retain the native caller return, amount write and Hand-picked call gate. The intact `Game.Win` body reaches the native achievement flag write in 98 steps under its stated entry frame; the

complete literal member-call prefix takes 135 steps, including argument allocation and declaration setup through arbitrary declarative closure paths. The proof preserves every other heap object and retains the five following native effects, including notifications, counting and the Steam callback, in the pending continuation. Sixteen native source comparisons test callback order and failure cases with explicit diagnostic callback fixtures. The current generator phase, initialized entry correspondence and completion of the remaining achievement effects remain separate obligations. The stochastic source wrapper branches at actual free-random requests; explicitly seeded calls remain source evaluation. Its finite prefixes preserve elapsed time, have total mass one, and compose by resuming only pending work. Returned callbacks, pending computation, and missing model operations are distinct outcomes. A callback return is not itself game completion. For a checked successful finite source branch with d free draws, Lean proves a success-mass lower bound of 2^{-53d}. This theorem retains all other branches. A diagnostic tape must still be represented as a checked source branch, and retry uniformity needs a common bound valid at every entry state.

For an existing lump, a further source prefix executes the intact `canLumps` call and return, obtains `Date.now()`, subtracts the stored lump timestamp, and writes the local age. In 69 interpreter steps it reaches the preceding age guard without a free draw or a change to any existing object. Composed with the choice branch, it reaches the free-draw request in 144 steps. The age is derived from the clock, rather than supplied as an independent branch input. Forty-eight source comparisons cover 36 native cases and 12 unbounded-clock cases. The two preceding animation calls, initialized frame and global generator-phase invariant remain separate obligations.

The five- and six-element native choice calculations also have universal arithmetic proofs. Put $N = 2^{53}$, $m = (N-2)/6$ and $q = (N-2)/5$. At an IID53 boundary, the golden-lump multiplier G from the native array $[2,3,4,5,6,7]$ and the meaty-lump multiplier M from $[0,0,1,2,2]$ have laws

$$\left(\Pr(G=j)\right)_{j=2}^{7} = \frac{1}{N}(m+1,m,m,m,m,m+1), \qquad \left(\Pr(M=j)\right)_{j=0}^{2} = \frac{1}{N}(2q+1,q,2q+1).$$

The native product rounds before the floor operation. Three bucket boundaries therefore differ from a real-arithmetic floor calculation. The exact means remain $\mathbb{E}[G] = 9/2$ and $\mathbb{E}[M] = 1$, but the individual probabilities are not uniform over the source arrays. Completion-time optimization needs the full laws. The proofs connect these masses to the actual binary64 multiply and floor for every free-draw index; 2 055 native source comparisons check the selected boundaries and samples. The generated source arrays are retained. Generator-phase preservation and composition with the complete harvest callback remain separate obligations. Under the selected zero-argument seedrandom restoration rule, a further source proof executes the final `computeLumpType` statement and its implicit return in 14 interpreter steps. It restores the captured free generator, preserves other objects and environments, and consumes no free draw or elapsed time. Fifteen focused tests include twelve intact `computeLumpType` calls and thirteen native comparisons under that same selected rule. This proves an endpoint once reached; it does not establish that every admitted callback reaches the reset or that vanilla seedrandom restores the same function identity. At the other end of the callback, 32 source steps construct the exact seed argument `Game.seed + '/' + Game.lumpT`; step 33 invokes the captured source closure and allocates its mapped arguments and lexical environment. The timestamp uses the selected successful clock codec. Twenty-nine checks include 27 ordinary native comparisons and two extended-clock arguments. From the prepared library state, a further source proof executes declaration setup, argument selection, and the actual flatten call and return. For every ordinary UTF-16 string seed, the flatten helper returns that seed

while retaining its allocations. After 128 builtin-machine steps, the real mixkey call has the seed and newly allocated key array as arguments. Original object properties, elapsed time, and free-draw count are preserved. The tagged setup proof now composes the 128-step prefix, captured source invocation, and complete mixkey body below. The general initialized-library entry and intervening `computeLumpType` branches still separate this prefix from the reset theorem. The selected timestamp-text extension now runs through the retained source library and save-codec helpers. The captured mixkey activation uses an exact traversal index; its stored source text and remaining statements are retained. Kernel statements prove the actual index increment, exit comparison, decreasing remaining length, low-bit reduction and character read. Thirty-one finite native comparisons check the complete seeded path, including an explicit 65 536-digit timestamp seed; eight more check actual save-codec round trips. Ordinary Date-prefix comparisons and extended explicit-text generator comparisons are separate claims. The 32-step timestamp argument construction and 33-step source entry now carry the text tag. The tagged flatten and setup paths now also have source proofs. A complete captured-mixkey theorem executes the retained UTF-16 loop, native key-array writes, the actual string-return helper, native Function.apply and String.fromCharCode, and return to the original caller. For every finite seed length N, it takes exactly $88 + 93N + 2\min(N, 256)$ interpreter steps, leaves elapsed time and the free-draw count unchanged, and retains at most 256 native key bytes. Its premises describe the captured library, intrinsic descriptors, prototype paths and initially empty key array. The proof has no finite timestamp-length bound. The source-core setup and invocation chain is composed, with the native allocations retained. A further composed theorem executes the actual seed expression, invocation, argument selection, flatten call, captured mixkey traversal and return through the browser runtime in $250 + 93N + 2\min(N, 256)$ steps. It retains the browser state, native allocations and original Math object, and increments the captured-activation counter once. Ordinary economic Numbers and storage quotas are unchanged.

The actual ARC4 constructor loops now have a composed source proof. From an explicit empty state array and a normalized nonempty byte key, the identity loop, intervening statement and key schedule take 37 661 builtin-machine steps. All 256 initialization writes and 256 key-schedule iterations retain the native descriptor writes, modulo key reads, index updates and two swaps, including equal-index swaps. The original key bytes are retained. Thirteen native constructor fixtures and 104 generator calls agree with the executable interpreter. The next composition derives the fresh instance, mapped arguments, declaration hoisting, all variable initializers and the empty-key normalization from the actual constructor dispatcher and caller heap. After dispatch, execution through both loops takes 37 738 steps for a nonempty key or 37 753 steps for an empty key. The proof no longer assumes a normalized nonempty key or an initialized state array. The complete constructor proof now includes its original invocation dispatch, the nested generator allocation and property write, the actual `g(256)` call, all discarded output reads and swaps, the saved generator indices and the constructor return. It takes 72 669 source-core steps for a nonempty key and 72 684 for an empty key. The returned fresh instance and all allocations remain in the proved state; no generator result is supplied as a premise. The native binary64 accumulator is retained, including overflow in the ignored 256-byte result. A further source-core composition consumes the retained mixkey return, writes its real binding and executes the actual `new k(j)` call through this constructor return. That segment takes 72 681 steps for nonempty input, or 72 696 for empty input, and leaves the actual `q` assignment pending. The next composition below executes entropy mixing and random-function installation under linked heap-frame premises. Seeded floating-output termination and general initialized entry remain separate obligations.

A further source-core composition executes the pending `q` assignment, reads the retained `q.S`, converts those 256 bytes with the actual `String.fromCharCode.apply`, mixes them into the existing entropy pool, installs the original random-function closure and returns `p` to its caller. It retains the closure environment, constructor state, pool writes and all source allocations, with no elapsed-time or free-draw change. The theorem supplies no helper return value as a premise. A canonical factory now derives those linked post-execution frames from the original constructor/library frame, a nonempty original entropy pool, intrinsic lookup facts and a writable original Math.random property. Thus no future helper frame is supplied. A proof-returning inspector now checks the original frame directly from a captured source heap. It validates the retained closures, lexical paths, byte pool, intrinsic functions, native Math receiver and writable random property. A finite trace checker then proves the browser-runtime dispatch agrees with each source-core step through the complete setup return.

The actual post-main source capture passes both inspectors. The tested ASCII seed takes 99 991 transitions after invocation dispatch, and the full direct-call theorem adds that dispatch step. A non-ASCII seed test also checks the returned original `p` against the pinned native engine. Two deliberately damaged fixtures are rejected. These are private diagnostic computations and do not change the continued game state. Full startup reachability, the later `Math.seedrandom` property lookup, and universal seeded-number termination remain separate obligations.

The actual ARC4 constructor now has a permutation invariant through initialization, key scheduling, the discarded prefix and return. A further source theorem maps each generator byte step and finite traversal to the exact byte-state transition, including the post-swap output index. That transition has a proved inverse. A complete finite check over 256^{258} output bytes is equivalent to all future output being zero; one nonzero witness instead gives a nonzero byte within that bound after every offset. This bound includes all byte tables and is not a practical enumeration result. For every injective byte table with its zero byte at slot zero, a further theorem supplies an eventual nonzero output. The same conclusion holds for any orbit that reaches this class. Thus an all-zero permutation orbit must avoid this class at every step. This does not show that every native orbit reaches the class. The classifier is total, but no theorem yet proves that every required game seed passes its nonzero branch. A separate recurrence theorem now proves finite normalization from a nonzero output witness. It retains native binary64 addition, multiplication, division, comparisons and unsigned shifts. Its denominator remains a native bit pattern, including overflow to infinity and a possible zero result. An all-zero future stream with zero numerator and extra byte never returns at any finite recurrence bound. The complete source closure now composes the initial six-byte call, each retained generator call, both normalization loops and the final caller return. A retained $g(n)$ call takes $136n + 93$ source steps; the initial $g(6)$ call takes 939. The proof keeps the actual constructor environment, state-array writes, function receiver, native denominator overflow and all call allocations. Its termination premise remains an eventual nonzero byte on that exact ARC4 orbit. A further proof-returning constructor reads the complete setup result and derives the draw frame, including the installed random property, closure scope, library constants, state array and generator method. Four source steps read that property and invoke it with the native Math receiver. The actual post-main capture passes this constructor. This finite acceptance test does not prove acceptance for every seed or remove the orbit premise. A further finite evaluator searches an actual retained byte prefix and computes the growth and shrink source frames. On success it returns the concrete final machine and binary64 bits, together with an exact source-execution equation for every remaining fuel value. It takes no completed random output or source trace as a premise. Three captured cases—an ordinary non-ASCII string and the selected extended timestamp renderings 8 640 000 000 000 001 and

$(2\cdot10^{40}+1)/2$—construct this result, retain a usable generator frame, and agree bit-for-bit with the pinned native library on identical text. These finite witnesses do not establish universal seeded-loop termination. The classifier and source theorem therefore supply neither a proper full-game route nor a finite expected completion time.

An unbounded integer UTC Gregorian calendar now has kernel-checked 400-year identities and selected boundary dates. Tests compare 5 408 native finite-range cases and 48 large-period extension cases. The source Date wrapper passes 24 finite-prefix checks. A separate timestamp-value module passes 499 arithmetic and codec cases, including 405 native-prefix comparisons. Its checked dyadic normalization gives unique text for equal extended timestamps, preserves fractional values beyond the native range, and proves that extended integer timestamps advance. The interpreter now carries exact extended timestamps separately from ordinary binary64 numbers. Selected source annotations cover eight save readers, the gift timestamp reader, and six zero timestamp initializers; removing the annotations reproduces the pinned syntax artifact exactly. Actual source checks cover 59 clock and storage cases, 54 save/load cases, 103 calendar cases, and 21 gift-code cases, including dates of magnitude 2^{1200}. Three malformed or unannotated source artifacts are rejected. Kernel proofs connect exact timestamp successors and differences to the actual evaluator and Date dispatch, while ordinary arithmetic retains binary64 behavior. Complete propagation invariants and full source refinement remain open.

The year decoder now has a universal interval proof for every integer UTC day. It follows the actual nine-step search, not a sampled-year table. A source-checked generator retains all eleven Easter formulas and the five seasonal guards in their original order. On these integer formulas, each required natural season contains two consecutive dates outside earlier guards. In particular, Business days mask three dates in common years and five in leap years. From any later rational time, a precommitted daily grid reaches a suitable window in less than 733 days and leaves one full day of slack. The saved-season field formulas also retain the nonpositive timer invariant after a planned ordinary reset and no later paid season switches. The bound is sufficient, not optimal. Generic binary64 proofs now show that the actual divide-and-floor expression gives $\lfloor n/d \rfloor$ for $0 \le n < 2^{40}$ and $0 < d < 2^{40}$. Exact native subtraction and the source day-length denominator connect the ordinary Date difference to its day ordinal. A further proof connects all eleven native Number computus expressions, in source order, to the integer Easter month and day for every nonnegative ordinary Date timestamp. The native constructor argument $M - 1$ is included. Generic arithmetic covers the years; only the 21 possible negative epact-minus-weekday pairs use a finite kernel calculation. These arithmetic results use the interpreter's UTC configuration and do not prove equivalence to a native local-time calendar. Source Math/Date bindings, constructor and getter execution, extended timestamp propagation, maintenance launch timing, and complete load/expiration callbacks remain separate obligations. The day of slack is not an assumed reload-latency bound.

The completion projection now reads required fields directly from the interpreter heap and applies the pinned full goal list. Unknown host fields, accessors, invalid counts, and noncanonical catalogue entries remain explicit projection errors. An inductive access-history relation starts empty and records only observed bought or unlocked flags. A checked theorem gives an earlier observed witness for every recorded access; another connects a successful heap check to every required goal. Tests cover 91 source-heap fixtures and one actual new-save data projection. The full transition system must still preserve the data-field invariants. The private monitor described below also retains transient access inside a callback, without exposing it as a player observation.

The browser model stores DOM nodes, handlers, resource requests, and timers explicitly. Its strict HTML parser passes 9 182 native comparisons, including the pinned index tree.

Ordered page and load-task diagnostics expose missing host operations. The empty-language web diagnostic returns from the actual `Game.Init` call and enters the first-launch language prompt, whose text layout remains unresolved. A separate configured-English diagnostic matches the language setting of the existing adapter. It returns from initialization, the first Logic/Loop callbacks, two Draw calls and the language-loader callback; later tasks drain the twelve pending images. Both diagnostics retain the web `App=0` branch. They do not execute the external Steam wrapper or establish full startup refinement. These diagnostics use an explicit environment fixture; they do not verify the external Steam wrapper or parser-blocking document visibility. Hashes and native metadata for 298 images and 81 audio files support the resource adapter without redistributing asset bytes. Metadata alone does not prove decoding, canvas rendering, or resource-event order.

Source closures retain exact UTF-16 function text. The native catalogue also records the text of 3 167 callable objects independently of their mutable names. This permits the loader's function-to-primitive comparison to execute through the ordinary interpreter. The Lean lexer and parser pass 358 checks, including structural comparison of all 40 pinned source units and 240 generated precedence cases. The dynamic constructor uses this parser and passes 44 checks, including 40 native comparisons. Parameter conversion, strict-mode validation, and wrapper text follow the tested native order. Unsupported modern parameter forms and legacy active-call reflection remain explicit. These tests do not prove general parser equivalence. Canvas dimensions follow the pinned integer conversion and attribute rules. The 2D-context adapter preserves tested option-getter order, cached identity, and the pinned macOS context attributes. Its drawing-state stack, native numeric, keyword, color and fixed-font setters, and dimension resets pass 491 checks, including 484 native comparisons. The model retains clear operations separately from the saved drawing state. A checked identity proves that clearing a blank bitmap preserves its initial pixels for every transformed clipping mask. The state proof also checks save/restore and reset identities for matrices and parsed font families. Affine operations pass 1 489 native comparisons and another 261 source-call checks, including 257 native comparisons. These retain binary32 input clamping and binary64 operation order, including singular matrices, zero-sized canvases, and reentrant argument conversion. Nonzero native C-library rotations remain explicit. Font parsing passes 221 cases, including 202 native comparisons; color parsing and serialization pass 7 203 cases, including 7 191 native comparisons. A further 82 rectangle checks include 79 native source comparisons. Typed deferred paint commands retain normalized geometry, matrix, font metadata, exact color bytes and drawing fields. Zero-area rectangles cannot all be erased: native copy and source-in composition can clear existing pixels. Pixel rasterization, readback, paths, general font resolution and metrics, context loss, and unsupported color grammar remain separate boundaries.

Source-function metadata now distinguishes explicit and inferred names without adding a false lexical self binding. The interpreter retains configurable function length and nonconstructible accessors. Strict functions use the shared throwing properties; active caller/arguments reflection of sloppy functions remains unresolved. These boundaries no longer block ordinary inherited call/apply. The 74 checks include 64 native comparisons. Non-strict arguments objects now retain the actual callee data property; strict arguments objects use the captured native throwing accessor. Complete invocation proofs preserve these descriptors and restore the caller. Unknown direct-call identities remain explicit.

The shared event dispatcher passes 79 checks, including 77 comparisons with the pinned browser. These cover registration order, capture and bubbling, mutation during dispatch, callback exceptions, nested events, cancellation, once and passive options, and object listeners. Nine further native comparisons test resource-load integration with the same dispatcher.

These checks do not prove the browser's general event semantics, native file scheduling, or image decoding. Runtime inline handlers use the Lean parser and the element/document/global lookup path. Their 45 checks include 40 native comparisons, vendor event-name aliases, and explicit unresolved form-owner, named-member, and Symbol cases. Native user-input targeting and default activation remain open. A further script-generated MouseEvent fragment runs ordinary HTML listener paths with a null related target. A click can finish only when the actual callbacks cancel its native default; trusted pointer delivery remains unresolved. Thirty checks include 23 native comparisons. A fresh-audio paused-state fragment has sixteen checks, including ten native comparisons and explicit parser/playback boundaries. Parsed autoplay and later attribute removal retain conservative playback provenance. In a synthetic null-view mouse sequence, the actual cookie handlers produce one cookie and one recorded click, retain the sound request and 48 consumed free draws, then stop at the native volume setter. This is a source-handler diagnostic, not proof of physical targeting, page coordinates, media playback or an admissible full-game action. The intact `PlaySound` source now has separate proofs for numeric and omitted volume when the sound setting is zero. Thirteen source comparisons retain getter, exception and music-branch effects. A fresh isolated browser run opens Options, sets the native volume slider to zero, and clicks the cookie; all seventeen captured input events are trusted. That finite witness omits the Steam wrapper and selects English. The first Options click can trigger sound before muting, so its audio transition and input/resource ordering remain open. Textarea value and reflection checks pass 157 cases, including 156 native comparisons. Parsed text supplies the initial export value; direct text-child changes update nondirty controls, and JavaScript value writes preserve the native dirty flag and line-ending rules. A separate selection suite passes 82 cases, including 80 native comparisons. It retains trusted select events in the native animation-event queue, freezes each explicit dispatch batch, and defers events queued by callbacks to the next rendering opportunity. A further 81 focused-selection checks include 65 native comparisons. The model distinguishes the cached textarea range from the focused frame selection, restores the cached caret before focus callbacks, and keeps selection-change tasks separate from animation select events. Rendering cadence, cross-category task order, caret scrolling and range changes from unmodeled DOM mutations remain explicit environment or host requirements. Pending events are not silently skipped; active animation and selection tasks cannot expose an early policy boundary.

The Storage adapter has explicit origin permission and quota inputs. The pinned Electron patch raises the limit to 100 MiB, measured as two bytes per UTF-16 code unit in keys and values. Native checks test both storage areas at the exact quota boundary. Source checks cover identity, conversion order, receiver validation, stored values, and the game's storage wrappers. The Storage adapter passes 37 checks, including 30 native comparisons, and retains its captured prototype if a script replaces the global constructor. Native multiple-key iteration order and exotic reflection remain unresolved. A separate Cookie adapter passes 33 checks, including 32 native comparisons, for the pinned file origin and explicit opaque-origin errors. DOMRect construction and reflection pass 215 checks; these do not supply layout or hit testing. These tests use temporary profiles and do not access the player save.

A typed CSS layer now derives a closed subset of layout from the two pinned stylesheets, connected links, live element fields, declaration state, and an explicit viewport. Selector, cascade, and integral positioned-box tests pass 328 cases, including 292 native comparisons; another 97 checks exercise the Browser adapter. No native rectangle is supplied to the calculation. A closed Lean CSS parser also handles runtime stylesheet text, including the actual launch-time background rules. The runtime parser now retains decimal and exponent tokens, balanced function values and supported pseudo syntax, including the Pantheon stylesheet. Its 473 checks establish parser equivalence for the selected syntax; they do not establish dynamic

selector matching or layout. Native opacity storage and text serialization pass 340 cases, including 336 pinned-engine comparisons, 100 internal numeric-value checks and 20 actual lump-icon source cases. The stored specified value is not clamped to the computed-opacity range; computed style and rendering remain separate. A kernel-proved compiler substitution also speeds up complete for-in fingerprint comparison. It compares every field and is equal to the previous Boolean guard, with no fingerprint cache or relaxed mutation check. A horizontal used-width solver also passes 145 cases through both its pure and live Browser interfaces, including 119 native comparisons. It handles a guarded integral box fragment with padding, borders, margins, bounds, and exact percentage calculations. A vertical used-height solver passes another 145 cases through both interfaces, including 107 native comparisons. It handles the two actual first-frame canvas height assignments under explicit positioned-box premises. Hidden ancestors are checked before unrelated descendant box grammar. A checked cascade condition permits paired hidden/visible overflow without resolving hover state. Transform declarations pass 2 157 value cases and 79 browser cases; their numeric cache retains more precision than native CSS text. Transform geometry remains separate. Fractional background-position parsing and serialization pass 161 checks, including 154 native comparisons; the internal numeric value retains precision beyond the displayed CSS text. The model retains nonzero canvas rotations as ordered native operations and clears the numeric matrix cache. It does not substitute JavaScript trigonometry for the platform C library. A border parser covers all 25 border declarations in the pinned stylesheets and passes 110 cases, including 100 native geometry comparisons. Native `border:none` retains the medium width value for a later style change; it does not reset that width to zero. A checked transition parser accepts the source opacity easing without admitting geometry-changing properties. A further canvas adapter records immutable image-source scenes and drawing state after source checks and ordered argument conversions. Its image-call suite passes 78 checks, including 61 native comparisons; 39 transform-history checks include 34 native comparisons. A byte audit admits 245 of 246 pinned PNG files and all 46 pinned JPEG files under explicit animation and orientation guards. Loaded-image replacement, responsive sources, unsupported image types and native DOMException objects remain explicit modeling boundaries. Source origin information is retained; no pixel value or origin-clean result is fabricated. The configured-English diagnostic now finishes 180 queued tasks, reaches `Game.T=155`, and returns with no pending continuation. Its fixed 4 096-value diagnostic tape supplies 181 free draws. Native backend return, save/restore realization, decoding, allocation and raster semantics still require refinement. General layout, layout-triggered events, hit testing, and unsupported stylesheet grammar remain open. Mouse and keyboard event fields pass 160 checks, including 156 native comparisons and ten actual source callback slices. These fields do not yet establish trusted input targeting or default activation.

The focus adapter passes 96 checks, including 87 native comparisons, with nested callbacks and native active-element fallback. Default scrolling, selection changes from DOM mutations, and focus loss from DOM or style mutation remain explicit boundaries. Element size metrics pass 80 checks, including 64 native comparisons and six actual source statements. Window naming passes 40 checks, including 39 native comparisons. The native global accessor survives variable hoisting, and its initializer then invokes the setter.

The timer adapter follows the pinned Chromium 87 source: signed 32-bit delay conversion, a minimum delay of 1 ms, and a 4 ms minimum when the incremented nesting level is at least five. Repeating timers schedule their next task before the callback. Thus the source's 1000/30 ms loop request becomes 33 ms. Lean checks active timer-ID bounds and freshness for the explicit coordinator. A separate theorem places accepted Number-valued waits on the 2^{-1074} ms lattice; it neither restricts arbitrary external policies nor bounds finite zero-

time action sequences. Cross-category task order, legal controls, public observations, and full source refinement remain open.

The proposed new-save funding route also has explicit arithmetic bounds. With scale 2^{59}, the native 0.01 literal has numerator 5 764 607 523 034 235. For the corresponding rounded-integer recurrence $z_0 = 2^{59}$, a kernel proof gives

$$\min\{2^{59} + n2^{50}, 2^{106}\} \le z_n \le 2^{106}.$$

Consequently, that recurrence reaches the multiplier 2^{47} by 2^{56} finite additions. The native arithmetic bridge checks both this result and the earlier, much slower Cursor-only bound: repeated native 0.1/30 from zero reaches at least 10^9 within $512 \cdot 10^9$ additions. Native fixtures check the initial 15-click/no-upgrade prefix, the challenge and ripe-lump boundaries, and a funded 1 024-cycle, 400-click burst.

The improved candidate opening buys one of each building through Wizard tower, in source order, with no upgrades and only 15 clicks. Both Neverclick and Hardcore are eligible before the first ascension, as well as in Born again. Owning one Grandma does not start the Grandmapocalypse without research. In the stated no-upgrade, zero-level source fixtures, the successive total CpS values are 0.1, 1.1, 9.1, 56.1, 316.1, 1716.1, 9516.1, and 53516.1. The fixed credit in each stage is the actual binary64 division of that total by 30.

Last purchase	Next bank target	Gain per credit, at least	Credit budget
Cursor	100	2^{-10}	102 400
Grandma	1 100	2^{-6}	70 400
Farm	12 000	2^{-3}	96 000
Mine	130 000	2^{-1}	260 000
Factory	1 400 000	2^{2}	350 000
Bank	20 000 000	2^{4}	1 250 000
Temple	330 000 000	2^{7}	2 578 125
Wizard tower	1 000 000 000	2^{9}	1 953 125

Lean proves the credit bounds for actual native addition with arbitrary normal-or-zero bank residues. Exact dyadic alignment permits the bank and credit to have different denominators. Below 2^{31}, one rounding loses at most 2^{-23} cookies. Affordable purchases at prices of at least 64 preserve a zero or normal bank; near cancellation cannot create a subnormal result. The first crossing of each target and the following purchase retain a bank bound of 2^{30}. The eight credit budgets sum to 6 660 050. A joint counter theorem starts with zero bank and 15 lifetime cookies, credits both counters, and debits only the bank. Round-to-nearest is monotone and commutes with exact power-of-two scaling. These properties preserve lifetime earnings at least as large as the bank, including purchase cancellation. The same finite route therefore passes the native lifetime-earnings comparison with one billion. The intact `Game.Earn` and `Game.Spend` bodies preserve this invariant under their explicit outer-environment frames. Initialized entry, source production, native purchase admission, and intervening callbacks remain separate obligations.

The native loop arithmetic also has a finite-prefix certificate. Chromium's timer conversion turns the requested native 1000/30 milliseconds into 33 milliseconds. With zero host latency, the binary64 latency credit is negative; repeated additions and the native clamp leave accumulated delay nonpositive. Lean executes the source catch-up statement and checks that it is skipped on every represented prefix. The source timeout branch is also skipped under the fresh-save preference. Conditional on the initialized scheduler and complete callbacks, eight

gains-update ticks and 150 final-check ticks give 6 660 208 ticks, or 219 786 864 milliseconds. The diagnostic's 200-millisecond readiness wait and fifteen 20-millisecond click gaps bring this to 219 787 364 milliseconds, about 61.05 hours. Readiness at that time is still a source-entry premise. The intact long game trajectory and a uniform finite-expectation completion route remain unproved; this is neither an optimal opening nor a full-game certificate.

The intact `Game.Has` source call now has an arbitrary-heap frame theorem through the actual declarative closure path. The interpreter executes argument allocation, declaration setup, the body and return in 67 steps. In normal ascension it returns the selected upgrade's actual bought value. All prior objects and the caller continuation are preserved; the new arguments object and local environment remain allocated. Successful property lookups, for-in enumeration shape and old local bindings have separate preservation theorems across these allocations. A further source-expression theorem executes the actual `Game.Has` property lookup and literal argument in 12 steps before this call. Initialized closure correspondence remains an obligation.

The complete `ComputeCps` source call executes in 55 steps with zero multiplier and bonus. It calls the native Math.pow model and retains the caller's argument allocation. Its Cursor specialization returns the exact binary64 literal 0.1. The complete `magicCpS` call returns one in nine steps. A generic lexical-path theorem resolves global data bindings without fixing local indices or unrelated bindings. The complete `Game.GetTieredCpsMult` source member call now also has a composed proof for the zero-upgrade opening. It includes the actual argument read, invocation, declaration scan, both full native enumeration loops, the fortune and Grandma guards, and the return to the original caller. Every executed `Game.Has` call retains its real arguments object and lexical environment. The proof returns native one and preserves the original object graph under explicit catalogue, zero-ownership, lexical-path and enumeration premises. Those premises still require initialized-state and automatic-callback preservation proofs. The eighteen complete source callbacks from Farm through You now compose both nested multiplier calls, their native compound assignments and their final return. The proof includes the original `me.cps(me)` call expression and all helper allocations. For a canonical positive normal base-CpS value, multiplication by the returned native ones preserves that value exactly. The complete `Game.eff(name)` call also returns native one in 58 source steps when its default argument is omitted and the named effect is absent, with native lookup, allocation, declaration scan and return retained. The complete `Game.auraMult` source call returns zero in 161 steps when both selected auras are the native "No aura" entry and the requested name differs from that entry. It executes both guards and retains native argument and local allocations; the second guard skips the dragon-level test through the actual short-circuit semantics. These call results alone do not establish source initialization or preservation across all of CalculateGains and Logic. A source inventory reduces the twenty production callbacks to three distinct bodies. The complete Cursor and Grandma source calls now also have composed proofs. Cursor executes all 22 statements, all 20 building rows and every nested helper, and returns the exact native 0.1 on the eight opening amount boards. Grandma executes all 18 statements, both enumeration loops and every helper call, and returns native one. Both proofs derive the final arithmetic from the executed source and retain every allocation. The complete `GetHeavenlyMultiplier` call returns zero in 902 steps with all eight Has calls and the aura call retained. The missing-buff call returns zero in 42 steps. The empty mod-hook call returns its input through the native array-length check and retains its argument allocation. These results still require initialized catalogue reachability and composition through the surrounding calculation and achievement-notification callbacks. Cursor's initialized `baseCps` field is a function; its callback uses the literal 0.1. Only the other nineteen `baseCps` fields are numeric values.

The nineteen numeric building-initialization cases now have exact kernel-checked binary64 results. Each proof executes the numeric test and three original CpS statements in 189 interpreter steps, writes the resulting `baseCps` descriptor and transient `digits` binding, and retains the nine following price statements. The resulting values are positive finite normal numbers. The installed native engine agrees on all nineteen cases. Separate field-creation and Cursor-skip proofs preserve the function-valued Cursor field. These are constructor-prefix results under explicit native entry and descriptor premises; complete constructor invocation and initialized-state reachability remain open.

The complete no-upgrade `Game.mouseCps()` source call executes all 48 statements in 4 400 interpreter steps and returns native one on the complete twenty-building opening catalogue. It retains both loops, all helper calls, allocations and caller state. A property-mutation frame preserves differently named lookups, prototype traversal and enumeration when a writable cache field changes. The complete achievement-count helper returns its native Boolean result for every string pool. The complete default-English achievement-label call executes in 227 steps, retaining its two function calls, four object allocations and two environments. Native queue proofs cover bounded dense-array unshift and pop, including their descriptor writes and return values. The first post-reset production-cache assignment also has a new-property proof; it does not assume that the cache already contains the key. These results support composition of the gain calculation; the complete cache loop and new-achievement notification effects remain required.

A universal bank-cycle proof lowers the earlier sufficient funding threshold. For every canonical finite bank in $[2^{58}, 2^{1023})$, adding any integer from zero through four or subtracting any integer from zero through fifteen preserves the native bit pattern. The proof includes subtraction across the lower binade at powers of two and every finite repeated-cycle prefix. The native Cursor formulas give a purchase cost of fifteen and a refund of four without free Cursors, or three with Starter kit. The intact getPrice and getSellMultiplier source helpers now execute in a checked no-modifier profile. The composed getPrice call takes 1 304 source steps, including all thirteen statements and twelve nested calls in modifyBuildingPrice. Its explicit descriptor and helper-frame premises retain the no-modifier profile; the proof retains source allocations, caller context and every pre-existing object. Separate DOM, text, style and class-list frame theorems retain their actual writes. The native displayed reverse-price quote is a different value from the actual unit refund: with one free-covered Cursor it quotes four but the sale credits three. Initialized entry and preservation through refresh, rebuild, buy and sell remain required before these results form a repeatable source cycle. A separate source theorem connects the earned-cookie recurrence starting at fifteen to the intact lump-eligibility callback and proves its true result by the stated finite credit bound.

The price-call premises now come from one original source catalogue. A checked constructor follows the actual closure paths and derives all later Has, aura, buff, effect and Pantheon frames after the retained allocations. It reconstructs that catalogue after native economic property writes. In particular, changing the Cursor's bought field does not change an upgrade's bought field; this uses object separation as well as property names. Separate complete source-call proofs cover an already unlocked string argument to `Game.Unlock` and an already won string argument to `Game.Win`. They take 76 and 93 steps after call preparation, respectively, and retain their arguments objects, native locals and caller return. The Win call still writes its local `it` binding. These branches leave all earlier objects, elapsed time and the free-draw count unchanged. The recursive array call is now composed: the native expression `Game.Unlock([first,second])` takes 266 interpreter steps when both named upgrades are already unlocked. The proof retains the array allocation, both recursive calls and their argument and local allocations. With Cursor amount one and the Click achievement already won,

the intact 27-clause Cursor purchase callback body takes 700 steps; its complete invocation, declaration scan and return take 758 steps. One original catalogue, closure paths and receiver property determine all intermediate frames. Existing objects, elapsed time and the free-draw count are preserved. This callback is only one part of the full store purchase: the surrounding price, debit, amount writes, sound selection, refresh and physical-input callbacks still need composition with the sale and repeated-cycle invariant. The complete muted `PlaySound` invocation now also retains its real arguments object, declaration scan, source locals and caller return. It takes 99 steps with omitted volume and 109 with an explicit nonmusic volume. The source reads the actual zero sound setting before returning; it preserves every earlier object and environment, elapsed time and free-draw count. Global call-expression proofs also retain native function lookup and argument reads. These results do not remove the separate random sound-selection draws. A source-adapter diagnostic checks the catalogue after 256 store inputs in 64 supplied-entry cycles, including free-Cursor and saturated bought-counter cases. It observes 128 native free draws. This is not an initialized-state or repeated full-cycle proof.

The complete numeric `Game.Earn` and `Game.Spend` invocations now run from an original object array and original lexical paths. They take 43 and 24 source steps from invocation, including mapped arguments, declaration scan and the native undefined return. The two Earn writes are sequential: the second frame is derived from the first write. All numeric bit patterns remain allowed, with native addition and subtraction. The proof does not replace Game or Window with a fixed synthetic layout. The complete existing-buff `Game.hasBuff` member call takes 56 steps and retains both native property reads. Its original closure paths supply the allocated callee frame.

The existing Devastation conditional is also proved on the original heap. After the original locals identify the buff, sold amount and god level, its diamond, ruby and jade paths take 37, 46 and 55 source steps. The result uses native multiplication followed by native addition, so an absorbed multiplier increase remains absorbed. Every other property, previous environment, elapsed time and free-draw count is retained. The next write frame is derived from the actual changed heap. The complete original Pantheon call now derives its helper frames from the original captured minigame, slots, function closures and Game paths. For the specified diamond/no-aura profile it takes 192 source steps. The complete Godzamok conditional then composes that call, the buff lookup, both caller-local writes, the outer success guard and the diamond update in 312 steps. All later helper and write frames come from one original entry; the result retains three new argument objects and three new environments. Native multiplier and sold bits remain arbitrary. This theorem requires an original successful sale flag and an existing buff. It does not prove that the earlier sale established these premises.

A separate finite interpreter receipt executes all four intact source callbacks for one supplied-entry cycle: switch to sell mode, sell one Cursor, switch to buy mode, and buy it back. The prepared fixture has bank 2^{60}, one Cursor, a level-one Temple, the required initial flags, muted audio, diamond Godzamok and an existing Devastation buff. It preserves bank bits and elapsed time, restores the Cursor amount, increments the bought counter, changes the native multiplier from 1.01 to 1.02, and consumes two free draws. It retains 239 new objects and 191 new environments. The portable source generator reproduces the exact request bytes. This receipt is finite executable evidence, not a closed kernel evaluation of the whole initialized state, a native physical-input proof, or a universal cycle invariant.

The complete sum and reverse price-quote calls for the one-Cursor cycle now execute from that original catalogue. Their prepared invocations take 1,410 and 1,520 source steps, respectively. The calls retain the native modifier and sell-multiplier helpers, arguments allocations, original caller and earlier heap objects. They preserve elapsed time and the free-draw

count. Sixteen pinned-engine comparisons check the selected quote and helper values. Full sale, buyback, refresh, rebuild and native-input composition remain open.

The original amount-one sale invocation now reaches its first loop body in 139 source steps. This includes dispatch, mapped arguments, the complete declaration scan, all three initializers, both amount guards and the native loop entry. The scan retains the repeated declaration of i and yields the real 14-binding local layout. A separate 19-step theorem exits that loop after its completed first body, with the actual postfix update and failed next test. Earlier and later environments remain in place. Generic native property increment and decrement proofs retain arbitrary Number bits, descriptor flags and prefix/postfix results. The first two loop-header initializers now compose in 1 519 steps, through both complete allocating price helpers and the actual caller writes. The next source assignment evaluates native `Math.floor(price*giveBack)` in 28 steps. The successful-sale amount guard reads the original building property; its true branch and the first five economic statements take 105 steps. Those statements retain native sold and revenue updates, the `Math.max` earned-counter rule, and the building decrement. The complete second price assignment takes 1 311 steps. Separate catalogue transports retain the original Math, Game, upgrade and Pantheon paths through the actual binding and property changes. The remaining successful-block suffix takes 71 or 81 steps, depending on the native zero-amount short circuit, and returns to the original loop continuation. Its fields and locals come from one suffix entry. The no-aura Dragon Orbs conditional also retains both native count-guard paths: 31 steps when the count test fails, or 197 steps including the complete aura call when it passes. Neither path draws a random number. Complete composition of these stages, later sale effects, buyback, refresh and physical-input admission remains required before these results certify a repeated legal cycle.

A finite funding theorem now retains the native arithmetic of each sale, purchase, multiplier update and click. Let N satisfy $2^{22} \le N \le 2^{49}$, and let the initial representable bank b satisfy $256N \le b < 2^{59}$. The fixed scalar production profile has one of each of the first eight buildings, Temple level one and Plastic mouse. Its mouse base is the native value of 536.941. Starting with a fresh unit multiplier, apply N native cent increments and N cycles that credit four cookies and debit fifteen, then apply 400 native click credits. Lean derives the final represented bank b' and proves

$$b + 168N \;\le\; b' \;\le\; b + 4528N.$$

All intermediate rounding is retained. If $b < 512N$, at most two such finite episodes reach a bank of at least $512N$. This is a finite arithmetic construction. Source reachability of the profile, the first buff creation, accepted-click timing, passive-credit interleaving and preservation through the full store callbacks remain separate obligations. It supplies neither a complete route nor an optimality bound.

A separate Dragon Orbs control schema has a proved conditional exit bound. Its source-derived effect list starts with Frenzy or Clot and contains at most 18 entries. At most five designated IID53 draws force an eligible sale, break an existing chain if necessary, force the next eligible sale, bypass last-effect removal and select the first entry. Other draws can follow arbitrary finite adaptive continuations. Thus the schema exits within two sale/pop cycles with probability at least 2^{-265}. Survival through k such blocks is at most $(1 - 2^{-265})^k$, and every finite-prefix expected block count is at most 2^{265}. The native audit checks all 3 840 syntactic list paths and the threshold and choice arithmetic. Complete sale/pop source execution, native target legality, buff creation and entry preservation remain required. Reloads, ascensions, forced cookies and automatic-task interleavings have not been reduced to this schema; it is not a closure theorem for all same-time strategies.

A funded Garden callback experiment constructs both full seed logs, the sacrifice, all seven upgrade drops, a full mature plot, and 1 009 mature harvests. It uses 4 925 Garden ticks and 171 505 selected free draws. The exact word has mass $2^{-9\,089\,765}$ under IID53. An independent tape replay reproduces the result. The experiment assigns its initial bank and Farm level, then jumps between Garden deadlines without running the intervening full-game timers. Thus it is a finite subsystem witness, not a legal full-game schedule or a uniform retry bound. A completion policy must also establish funding and recovery on every retry history. A separate integer-rounding theorem bounds the persistent reset counter: if the prior counter is at most 2^{1000} and a retry contributes at most 2^{940}, its rounded update remains at most 2^{1000}. Induction extends this barrier to every finite retry prefix. The arithmetic bridge now connects these integer counter updates to the interpreter's binary64 operations. Source earnings bounds, fractional callback inputs, recovery, and invariants over the complete retry route remain proof obligations; the barrier alone does not establish a repeatable route.

The separate 136-lump prefix was checked with zero Grandmas, no lump-time upgrades, and the order deity unslotted. Extracted source checks, including intact calls to the actual upgrade, aura, and Pantheon helpers in three early layouts, give a 23-hour ripe interval and exclude the meaty type in the checked zero-wrath state. The nonmeaty source options give one through seven lumps. Universal arithmetic proofs check exact small-natural addition and affordable subtraction, and any 136 such credits supply the proposed 136-lump level budget. The interval calculation gives 11 260 800 136 milliseconds with one millisecond slack per interval. Full source composition, random-range, funding, and scheduling invariants are still required. This deliberately slow candidate prefix supplies no optimality claim.

The standard Steam startup path also has a source-derived restoration gate. Both branches register a timer with a requested delay of 100 milliseconds before calling `Game.LoadSave`. Lean executes the intact scheduling statement in an explicit entry frame and proves that actual modeled registration and firing retain the 100-millisecond lower bound at every timer nesting depth. Native ready state can precede save restoration, so it is not a substitute for restored state. The source audit finds no restoration in current-save export. Initialized Steam entry, full save loading, alternative entries and pre-load controls remain separate obligations; no proof covers every reload interleaving.

The executable adapter now supports precommitted maintenance reload schedules. It follows the native save/load path, retains the current save and continuing free-draw stream, and creates a new VM and timer queue. Native startup delay, Garden and market rounding, and temporary-effect loss are retained; reload is not an identity transition. Requests during loading or ascension are deferred. This is one permitted scheduling subclass, not a proof that every admitted maintenance strategy is represented.

The policy evaluator passes immutable observations to a separate JavaScript context. It withholds the engine handle, clock controls, and future random tape. The native Export save action exposes only the current visible read-only save text, which the policy may decode, including the startup seed. Reading an already open export does not serialize the game again. Native export bookkeeping is retained: export can clear a pending save request. If a later maintenance navigation would then restore a stale save, the evaluator rejects that trace as inadmissible. A native save request before navigation can restore the allowed path. It starts each experiment from a new save. It records completion after each native action and timer callback, including callbacks during a requested wait. The Lean trace checker verifies that the first reported complete observation satisfies the full predicate and has no earlier complete checkpoint. A later ascent or Garden sacrifice does not erase that earlier hit. This is a theorem about the supplied chronological trace, not a proof that the trace contains every legal transition.

The policy-interface formalization proves that identical public serialized input histories give identical decisions in its deterministic controller model. Serialized object order is retained. A separate native-click model proves that t elapsed milliseconds permit at most $\lfloor t/20 \rfloor + 1$ accepted ordinary clicks. The engine rejects earlier clicks; an arbitrary number of zero-time action calls does not give arbitrary cookie income. A finite-clock theorem combines monotone binary64 clock ranks with the evaluator's same-time action cap. Fourteen native clock and click projections are checked separately. This cap bounds an experiment, not every legal gameplay policy. The adapter also assigns no simulated time to policy computation or to a native action callback; real execution latency is an unresolved environment distinction.

At a finite experiment horizon H, an unfinished path contributes H, not zero, to the sample mean of $\min(T, H)$. An action failure or a decision-budget cutoff is not counted as a completed horizon sample. This prevents an unfinished simulation from appearing to complete quickly. The restricted sample mean is neither the unrestricted expectation nor a global lower bound.

A constructive Garden controller covers all 34 seeds, including spontaneous weeds, uprooted fungi, and multi-parent mutations. On the full plot it uses four disjoint eight-queenbeet rings for Juicy queenbeet attempts. Native-kernel regression runs complete the seed log, sacrifice it, rebuild it, and obtain all seven Garden upgrade drops, the harvest-count goal, and the full-plot goal. Those tests begin with a funded, level-9 farm and advance Garden steps directly. They do not establish a new-save completion time. The composed experimental policy reserves requested planting costs, preserves the initial-run Neverclick and Hardcore conditions, and requests further ten-lump sacrifices while finite level costs remain. It does not assume that the single required sacrifice supplies all 126 mandatory level lumps. A Farm level from 9 to 10 supplies a further normal achievement when the final milk would otherwise remain locked at 599.

Slow novel crops can receive elderwort support. With eight mature elderwort neighbors on fertilizer, the native age multiplier is $(1 + 0.03 \cdot 0.75)^8 \approx 1.19483$; with seedlings it is $(1 + 0.03 \cdot 0.75 \cdot 0.1)^8 \approx 1.01814$. Nursetulip and Juicy queenbeet power effects do not enter this age multiplier. At fixed tick times, with the same per-plant age and rounding uniforms and no contamination or premature death, the larger multiplier cannot delay maturity. This does not give an exact reciprocal reduction in expected hitting time. Elders themselves mature over time, and the cookie expense and changed random-stream consumption prevent a full-game dominance claim. The controller limits this support to four slow crops and avoids cells that also border another novel plant.

A further candidate phase collects Thick-skinned by direct veil trials. After the planned heavenly-upgrade, growth and spell-count phases, the scheduler makes one fixed final normal-run reset. It reuses the spell controller's final reincarnation only when that controller actually completed its fixed cast quota and final reset. The final-run preparation decision is independent of the seed, the veil result, and whether Thick-skinned is already won. No unsuccessful veil trial causes another reset. This phase starts with `cookieClicks=0` and all four veil-defense upgrades. It sells the starter buildings through ordinary store controls, waits for a native gains update, and requires observed zero production before each click. The native rebuy price, evaluated as `Game.unbuffedCps*60*60*24`, is then zero. The policy checks the visible zero-price offer before each purchase and uses a fixed 34-millisecond accepted-click cadence.

The zero-production condition needs a specific exclusion. The normal upgrade whose name is the quoted string `"egg"` adds 9 CpS without buildings. The candidate excludes this upgrade from permanent-slot assignments; it still buys the upgrade during final collection. Ordinary Reset clears nonprestige bought flags before applying Keepsakes, which can retain the corresponding unlock but does not retain that bought flag. The controller reports a

nonzero production effect as an unresolved phase boundary. It does not assume that selling every building alone proves zero CpS in an arbitrary save.

An exhaustive finite computation gives a candidate bound for the consecutive click trials. Native `ClickCookie` calls `loseShimmeringVeil` before incrementing `cookieClicks`, so the first trial after reset uses the explicit seed string `seed/0`. For each of the $26^5 = 11\,881\,376$ five-letter startup seeds, the archived computation selects a counter $c \leq 34$. At that counter, the first post-discard ARC4 output byte is between 1 and 101. This stronger byte condition puts the generated draw below the native four-upgrade defense threshold; the conservative byte bound is $102/256 < 0.4$. The computation also checks that the initial six-byte word is nonzero at every counter from 0 through the selected counter, so the checked prefix does not start any normalization loop with a zero numerator. The largest sufficient counter is 34, first found for seed `awjiu`. These are sufficient counters, not necessarily the first successful native trials. The controller reads neither the seed nor the certificate: it clicks consecutively, buys the free toggle when needed, and stops on the visible native award.

The archived evidence includes the per-seed byte certificate, source and build commands, all-seed statistics, and 6 192 exact first-output comparisons against the pinned native seeded library. A supplied-entry source fixture with the worst sufficient seed completes in 35 clicks and 34 free rebuys after its fixed reset. The composed controller tests supply prior milestones, heavenly ownership and funding, then execute the actual reset, spell, sale, purchase and click callbacks. A 62-test regression batch, including eight new tests, checks spell-boundary reuse, reset decisions with the veil achievement already won or missing, the quoted-egg exclusion, and scheduled maintenance recovery. Active spell and final-run phases retain their memory while waiting through the native deferred maintenance prompt; they do not confirm that prompt. These tests do not give a uniform reload-completion bound.

The finite seed certificate still needs a kernel-checked verifier and a source proof for ARC4. A full route also needs the source and physical-input bridge, callback termination, zero-production preservation, and a new-save proof of the phase's entry conditions. The diagnostic 35-click cover supplies no full-game expected-time bound, global lower bound, or optimal policy. The frozen long new-save experiment uses its earlier controller and is separate from this supplied-entry evidence.

After the required Garden work and final-run preparation, the candidate now grows golden clovers and Nursetulips while Black cat's paw remains missing. This normal achievement requires 7,777 clicks on timer-leading golden or wrath cookies, including chain leaders. Fortune rewards, Dragon Orbs, spell-created cookies, storm drops, and Distilled essence duplicates do not themselves increment that counter. Chains can shorten the next leader interval to 91 native updates at 30 frames per second. Untimed cookies can also alter the shared chain, so their pop order matters. A fixed multi-minute interarrival lower bound would be false. For mature plants on clay, a clover with k adjacent Nursetulips contributes $(3/80)(5/4)^k$ to the frequency multiplier. A row dynamic program compares every binary layout of the 6×6 plot. Its state retains two rows, so a third row determines the middle row's contribution. Multiplying all power sums by 4^8 gives exact integer computations. The maximizing layout alternates full rows of clovers and Nursetulips, with frequency multiplier

$$1 + \frac{3}{80}\frac{3\,356\,800}{65\,536} = 2.9207763671875,$$

compared with 2.35 for all clovers. Independent exhaustive checks cover smaller plots, and the native mature-plant effects agree with the full-plot witness. This JavaScript planner is not Lean-verified. Nor does the static maximum optimize planting, maturity, death, cookie costs, or the Nursetulip production penalty. The controller retains mature plants, replaces dead

plants, and reserves their native planting costs. It retains Elder Covenant after the required wrath goals because clovers do not increase wrath-cookie frequency. Native shimmer intervals are updated when the preceding shimmer dies; crop growth does not continuously rescale an already pending interval.

The corresponding Stock Market controller reaches all eight trading and office goals in three funded native-kernel tests with different random streams. It checks prices, overhead, capacity, trade restrictions, and the single-transaction thresholds of 86 400 market dollars. Debt evasion needs a separate coordinated ascent while a loan is active; its later interest period does not qualify. Reincarnation resets current market offices and profit. These tests demonstrate subsystem behavior, not an optimal whole-game allocation of time or cookies.

The native UI imposes further constraints. A hidden office button is not a legal purchase, and a loan must use its visible native control. The runtime and mod now enforce those conditions. There is also an unbounded-tail issue: the persistent binary64 draw counter can stop increasing at 2^{53}, leaving periodic visibility updates inactive. A proposed proper route therefore retains office level 4 through uncertain retries and buys level 5 only at its final deterministic stage. Store upgrade creation still runs through native logic at this counter value; previously visible product rows survive an ordinary reset, but rebuilding the store can hide them. These source findings constrain the candidate route and do not yet prove its uniform retry invariant.

The composed schedule now separates loan preparation from final collection. Before reset, seasonal and spell-count goals are complete, it stops market preparation at the second office, which opens the first loan. Brokers, holdings, and later offices are deferred. The market controller has no default lifetime decision cap. Season-switch costs grow exponentially within a run; after a bounded number of uses, the seasonal controller requests an ordinary reset. These changes remove identified ways for the previous candidate to stall or repeatedly lose expensive current-run investments. They do not establish a finite expected full-completion time.

Another market defect permitted repeated acquisition losses before Make my day: a holding bought to meet the single-sale threshold could be sold after its price fell, without earning the achievement. The corrected controller retains that holding until a sale qualifies. A native alternating-price regression checks this case. Separately, an exact-rational translation of specified native market updates proves an 801-tick low-price block and a 231-tick high-price block from explicit price and velocity bounds, for Bank level 1 and all permitted Supreme Intellect strengths. A typed draw plan uses at most 337,464 free draws for the pair and has positive finite-cylinder probability under the selected law. Native boundary comparisons cover all six modes and 18 goods. These results do not establish a general binary64 refinement, a uniform funded trading bound, or a full-game expected-time certificate.

The scheduler now includes native ascension, seasonal, and miscellaneous controls. Ascension tests purchase all 129 heavenly upgrades, whose undiscounted prices total about 7.49×10^{16} chips. The three Lucky gates count sevens anywhere in the displayed native prestige number. Large-number tests therefore use attainable binary64 targets and check the native gate after cube and cube-root rounding. The standalone ascension planner retains its growth-first schedule. The experimental completion candidate now requests 100 early profitable resets, each with at least 2^{90} current-run earned cookies and a positive native prestige gain. It permits normal prestige-activation purchases after the first-run challenge conditions and permits permanent upgrade assignments; neither profitable resets nor the Lucky gates require a general activation hold. It gives the first ripe sugar resource to the Temple income route and defers unrelated collection. A funded 100-cycle source regression verifies the reset ledger, but supplies each run's income premise; it does not prove that income from a new save.

Fixed one-second input batches preserve all native callbacks. Explicitly forbidding resets releases the other controllers. The final growth targets include 10^{72} cookies baked in one run, 10^{56} raw CpS, and 10^{31} handmade cookies; temporary buffed CpS is not a substitute for raw CpS. Seasonal tests cover hearts, Santa, reindeer, Halloween cookies, eggs, research, pledges, fortunes, wrinklers, and dragon drops. Prompt ownership, held pointer state, Fading luck, Eldeer, and reincarnation cleanup are tested in the composed controller. These tests still do not demonstrate a complete new-save trajectory or its expected time.

A separate final-run recovery controller now uses natural seasonal windows, dirt for Garden collection, and at most 1 000 shares per holding and transaction. It leaves golden cookies untouched after the counted-cookie phase, waits for existing economic buffs to expire, and defers office level five until the stochastic collection goals are recorded. Its host must commit to daily current-save maintenance before the new save begins. It retains controller memory through restoration and confirms only a prompt that it requested at the same elapsed time and menu. It checks the unchanged complete predicate before stopping. The entry guard requires all prior reset work, heavenly ownership, Farm level ten, Cursor level twelve, maximum dragon level, and the named earlier achievements. Fourteen new tests and all 48 existing subsystem tests pass; the full repository suite has 411 tests plus seven patcher tests. These checks use conditional snapshots and supplied-entry native callbacks. They do not establish entry reachability, a uniform recovery law, finite expected completion time, or optimality.

The completion candidate now invokes a separate bounded income controller. After both first-run challenges, Temple level 1, diamond Godzamok and Plastic mouse, it uses actual store controls for a fixed finite number of Cursor sale/buyback cycles. It then waits for a gains update and sends at most 400 native clicks over at most eight seconds. The default budget is 16 384 cycles. It uses one Cursor at lower banks, or 100 Cursors when the observed bank, other-building income and buff duration meet its stronger guards. It reserves native purchase debits, retains sale proceeds and checks the result after each batch. Existing Devastation buffs must also retain more than one second beyond the requested click span: additional sales change their multiplier without refreshing their remaining time. Between bursts, fallback growth buys other building types so the next burst does not sell its latest growth investment. Final building collection is separate. The scheduler owns the transaction until it returns. A scheduled current-save reload can restore the original saved buff argument instead of its later stacked multiplier and can discard clicks during loading. The controller retains that observed result and releases an interrupted burst. A native boundary test accepts 388 of 400 clicks after a precommitted reload; the composed controller then resumes a later burst. This is a recovery test, not a uniform reload theorem. It also withholds One mind until the first Temple level, to preserve the proposed nonmeaty first-lump route. These guards and supplied-entry tests improve the candidate; they do not prove its full-route return or optimality.

A separate initialized source-adapter sample uses the earlier controller with a fixed 10^{12}-cookie early reset target. It completes its first normal reincarnation at 20 820 634 milliseconds, retaining Hardcore and Neverclick and earning prestige 1. It buys Legacy and retains the original lump timestamp. Thus the earlier 30-hour zero-reset sample was not an unavoidable opening bound. This run uses the fixed diagnostic stream seed 1, not an IID Monte Carlo estimate. Its archived receipt contains the frozen policy, configuration, progress log and final observation; it records summary counters, not every action checkpoint. It supplies neither an independently checked complete replay nor an expected-time bound. The default reset target is unchanged: this sample alone does not compare the completion times of the two schedules.

The growth controller can configure an active Pantheon, fill empty dragon aura slots, and cast Force the Hand of Fate during an observed positive buff. It does not yet use public seed inference to predict the spell outcome. This is a restriction of that candidate, not of

all permitted strategies. Godzamok sales require a funded native buyback quote, exclude minigame buildings, and keep a copy of the sold building. The scheduler completes a sale and its buyback before another controller can ascend or spend the reserved bank. These local return estimates remain heuristic. Finite precommitted click sequences reduce evaluator overhead; all native inputs and timer checkpoints remain in the replay. They receive no intermediate observations, so batching is a distinct policy when an opportunity appears within a batch.

An exhaustive native-arithmetic computation supplies a finite helper-Grandma display bound. Before Grandma synergy artwork is unlocked, an active Christmas display has two artwork choices. For every one of the $26^5 = 11\,881\,376$ startup seeds, at least one of the first 24 seeded sprites selects the helper artwork. The worst first index is 23, for seed `ubyju`. A fixed 3 350-millisecond pointer sweep checks the public grid without inspecting a seed or private sprite coordinates; three native fixtures exercise the sweep. The archived certificate gives one index for each seed. Its stronger byte predicate is checked exhaustively by the compiled verifier, but a Lean proof of the complete finite cover and the rendering correspondence remains open. This bound can remove an indefinite helper retry from a completion route once the early two-artwork setup is reached; it does not prove that setup or the whole route optimal. The candidate now withholds the 18 artwork-changing upgrades and their permanent-slot assignments until the helper achievement is won. After an already scheduled profitable reset, it funds 24 Grandmas and the Christmas trigger, then runs the fixed sweep. It checks the display, animation, focus-related menu and prompt premises; a conservative elapsed-time bound applies only to its stated single-loop execution profile. A failed sweep is an unresolved route boundary, not a reason to choose another reset from the random result. The funded native setup tests do not establish first-reset funding from a new save.

Summary-only experiments retain final observations, counts, and source identities without allocating replay histories. Current recordings pin all ten model implementation and configuration dependencies and the Node/V8 execution environment in addition to the game files. Replays reject mismatched revisions, including changes to completion logic, its profile, or the mathematical assumptions. Replay and evaluation results explicitly leave strategy admissibility uncertified: action replay cannot establish whether a decision rule chose a timestamp to change a seeded outcome. The adapter exposes 62 named action types. Its 334-site source report retains 19 partial entries. Native big-cookie and tooltip pointer actions preserve the reviewed element and document handler order, mouse and crate state, incidental sound draws, and delayed hiding. Viewport tests preserve existing seeded sprite indices when the display expands. A kernel-evaluated short ASCII ARC4 witness agrees with the native helper-Grandma display, but does not verify a general browser renderer. The external Steam Music service, full layout, microtasks, and general tooltip caller coverage remain open.

A six-hour new-save sample under the revised schedule wins Neverclick and Hardcore. It ends with 57 normal achievements, 57 bought upgrades, about 7.50×10^{11} cookies baked, and 3.63×10^7 CpS. It has not ascended and still lacks 1,430 terminal conditions. The evaluator processes 72,875 decisions and 1,432,438 native checkpoints. This is a censored summary-only experiment, not a complete trajectory, an action replay, or an expected-time certificate.

A later experiment requested 24 hours with 50 clicks per second. It stopped at 51 170 068 milliseconds, after two ascensions, because a policy decision exceeded the one-second execution guard. It still lacked 1,389 terminal conditions and had harvested no lump. The run processed 173,415 decisions and 6,484,686 native checkpoints. The evaluator rejects it as an unscored failure, rather than a 24-hour censored sample. Its immutable source snapshot predates the startup correction. The wall-time guard is an experiment resource limit, not a restriction

in the zero-latency mathematical strategy class. This failed run supplies no completion-time expectation.

The source effect audit also prevents an unsound simplification of rendering. It checks the 50-record particle store, nine callsites and both returned aliases, and records the random draws and native calls of seven complete functions. Twelve guarded source fixtures exercise the recorded effects. Drawing can call `Pic`, whose cache miss schedules an image load; that callback can invoke startup through `Loader.loaded` unless `doneLoading=1`. An economic projection must prove this phase invariant or retain the callback, as well as all relevant random, clock and task effects. The inventory alone is not a simulation theorem.

A pinned-engine source experiment also rules out erasing fractional action times merely because `Date.now()` has the same integer value. Gambler's Fever Dream schedules a spell after 1 000 milliseconds. Against a configured 33-millisecond Logic schedule, a cast at 23 milliseconds can place the queued payment before the Logic callback at 1 023 milliseconds; a cast at 23.5 milliseconds places it after that callback. Native magic regeneration depends on current magic, so these orders give different resulting binary64 values. Lean checks their strict numerical difference and the equal integer-clock projections. This is a local source experiment, not a reachable new-save witness or a proof that the full-game infimum is unattained. A valid time abstraction must retain relevant callback order and prove that its minimizing choices are attainable. The actual modeled timer/resource dispatcher now has checked clock laws: it never reduces elapsed time, and a selected task advances to the maximum of current time and its deadline. If both are no later than the player's chosen deadline, the dispatcher does not cross that deadline. These laws retain resource preparation, Event allocation, callback setup and native queue selection; they do not prove a complete native input or scheduling refinement.

One closed physical-input path now has a composed source proof. A retained Escape-down device state and an unfocused ordinary HTML document determine the target and keyboard fields for the physical Escape release. The proof executes actual trusted Event allocation and dispatch, both native key-code getters and the intact game callback through return in 147 browser-runtime steps. It clears `Game.keys[27]`, copies `Game.time` into `lastActivity`, and copies `AscendDuration` into the active `AscendTimer`; elapsed time and the free-draw queue are preserved. A pinned-engine key-down then key-up test checks the same fields and callback effects in an isolated fixture. The Lean entry frame is conditional: source key-down, initialization reachability, arbitrary focused controls and general physical input remain open.

The proof boundary is explicit. The current action registry omits some legal UI interactions. No checked refinement yet connects complete game executions and browser behavior to the Lean transition model. A checked transport theorem permits a stronger lower-bound competitor to observe the whole current state and use a larger legal action set. It preserves the public strategy's startup law, admissibility and every capped expectation before taking the unbounded hitting-time limits. This reduction supplies no numerical bound or attaining policy. No full-game value function and attaining policy have been supplied. Thus no theorem in this project certifies the current mod as globally optimal for expected completion from a new save.

A controlled source kernel now composes the actual stochastic interpreter, timer/resource selection and rational wait deadlines. A committed wait continues through automatic tasks; an interruptible wait returns control after the next completed callback. Internal source steps and random draws do not enter the controller's observation history. The private decision machine also permits an unbounded number of private computation and random steps before an action. Genuine noncompletion and zero-time divergence cost infinity. An unresolved

model transition has no assigned numerical cost, so it cannot supply either a completing policy or a lower-bound shortcut.

A private completion monitor checks the source heap before each interpreter step. It retains access to the 17 selectors even when their flags change within one callback. Each retained positive bit has a checked source-descriptor witness. Known absent startup fields and a native achievement count below the threshold can exclude completion; the counter-gate proof establishes this implication for every successful full projection. Unknown fields and accessors remain explicit errors. A hit records the current elapsed time, preserves the heap and controller information, and persists through all longer cost prefixes. It creates no player action or observation during a callback. Ten source fixtures exercise transient completion, transient selector access, startup absence and unresolved getters; two pinned-engine heap comparisons confirm the corresponding transient predicate change. Full native source and host refinement, physical input delivery, scheduled maintenance navigation, task priorities, public observations, timing-policy admissibility and startup remain separate obligations. The kernel supplies neither a complete new-save policy nor an attained minimum. The native remainder operation used by the seeded generator now has a general proof for natural operands below 2^{52} and a positive divisor; it is not a finite table of byte cases.

Completion does not unconditionally persist to a callback's return. In a supplied late-game fixture with Seedless to nay already won, a full seed log and 999 mature harvests, a second Garden sacrifice can earn the last missing achievement, Green, aching thumb, during its initial `harvestAll`. The complete predicate then holds, including the native achievement counter. The same callback subsequently locks 33 required seeds. The isolated source adapter reproduces both states at the same elapsed time. Calling `harvestAll` without the sacrifice instead retains completion in that fixture. These checks do not prove new-save reachability or a general replacement-policy theorem. They show that checks confined to returned boundaries do not cover the full stopping objective.

The audit inspected installed Steam version 2.053, including `main.js` and the Pantheon, Grimoire, Garden, and Stock Market sources. The game installation and save were not modified. The repository audit record gives source hashes and regression evidence. Historical simulation claims remain outside the checked results.

11 Limitations

- The run curve is extrapolated from one number, the current growth exponent. Real runs accelerate when new tiers of upgrades unlock, and slow down when they run out. In simulation, the extrapolation of a decelerating run made the planner too patient by about 15% in its first runs (Section 8).
- The heavenly multipliers marked as estimates in Table 3 do not come from the game's code paths. Blocks restricted to ancestor closures are a heuristic for general precedence graphs.
- The golden-cookie probabilities π_F and π_L are approximate. Assumption A4 holds only approximately for golden cookies and the "Lucky!" reserve.
- The analysis is risk-neutral. The rules of Section 7 rest on the income model and on simulated outcome shares. Several heavenly values, the Garden's seed values and the Stock Market thresholds are estimates.
- The released mod's Garden heuristic covers only part of the seed catalogue. The separate experimental controller covers all 34 seeds and also has market and spell-count goals, but

it does not implement a complete new-save route. The play horizon of Proposition 7.2 is a prior, not a measurement.

- No full-game completion policy with a matching expected-time lower bound has been established. The necessary timing bounds do not prove attainment. A complete checked refinement of the installed game implementation remains open.
- The simulator is a stylised economy. Its agreement with the theory supports the internal consistency of the rules, not their exact optimality in the real game.

A Algorithms

Algorithm 1 Ascension planner (every five seconds)

measure A, C, the run time T and the income c; set $\beta \leftarrow \min(\max(cT/C, 0.3), 12)$
build the chip order of Theorem 5.1; define $S(P)$ by (6.7)
for each of five goals a_k^* spaced over one decade beyond the 10^4-run-length horizon **do**
    grid $\leftarrow$ 400 log-spaced levels in $[A + C, a_k^*] \cup$ whole prestige levels < 400
    solve V backwards by (6.9); choose the target j_k that minimises remaining$(j)+\tau+V(j)$
    consider only targets with a speed gain of at least 10%
end for
target $\leftarrow$ median of $j_1, \dots, j_5$; **ascend now** if and only if the target is the current level

Algorithm 2 Automation tick (every frame; clicks on a separate 50 ms timer)

click every golden cookie and reindeer on screen
if the store is in buy mode and the values are current **then**
    for each candidate i: value $\bar{\Delta}_i$ by (4.5)
    repeat
        reserve $R_i \leftarrow$ the "Lucky!" bank if $p_i/\bar{\Delta}_i > \theta$, else 0
        $i^* \leftarrow \arg\min_i \ \max(p_i - (b - R_i), 0)/(c + k + G) + p_i/\bar{\Delta}_i$
        buy i^* if $p_{i^*} \le b - R_{i^*}$, with the next copies of a building that pay back no slower than the runner-up
    until nothing is affordable, 4 ms have passed, an upgrade was bought, or a second purchase needs a reserve
end if
if the planner has said "ascend now" for 10 s **then**
    pop wrinklers; wait for payouts; buy the Chocolate egg; ascend; spend chips along the order; reincarnate
end if

References

[1] R. Bellman, *Dynamic Programming*, Princeton University Press, 1957.

[2] E. L. Charnov, *Optimal foraging, the marginal value theorem*, Theoret. Population Biol. **9** (1976), 129–136. doi:10.1016/0040-5809(76)90040-X.

[3] Cookie Monster+, https://github.com/JWKNT/cookie-clicker, guide at https://jehlp.net/cookie-clicker/.

[4] Cookie Monster Team, *Cookie Monster*, a Cookie Clicker add-on, https://github.com/CookieMonsterTeam/CookieMonster.

[5] E. L. Lawler, *Sequencing jobs to minimize total weighted completion time subject to precedence constraints*, Ann. Discrete Math. **2** (1978), 75–90. doi:10.1016/S0167-5060(08)70323-6.

[6] J. B. Sidney, *Decomposition algorithms for single-machine sequencing with precedence relations and deferral costs*, Oper. Res. **23** (1975), 283–298. doi:10.1287/opre.23.2.283.

[7] W. E. Smith, *Various optimizers for single-stage production*, Naval Res. Logist. Quart. **3** (1956), 59–66. doi:10.1002/nav.3800030106.

[8] J. Thiennot (Orteil), *Cookie Clicker*, version 2.053, DashNet, 2013–2025. https://orteil.dashnet.org/cookieclicker/.

[9] Ecma International, *ECMAScript Language Specification*, Math.random, https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.random.